

Cartridge & Cloud — Post-Launch and Live Operations Plan

Plan consolidado de soporte, incidentes, parches, comunidad, métricas, mantenimiento y fin de vida

VRM Games / Blas Luis Rocha González

2026-07-06

Cartridge & Cloud — Post-Launch and Live Operations Plan

Proyecto: Cartridge & Cloud

Desarrollador: VRM Games / Blas Luis Rocha González

Plataforma prevista: PC / Steam

Motor: Unity 6.3 LTS 6000.3.18f1 / URP 17.3.0

Versión observada: 0.0.21

Estado: Sprints 0–15 CLOSED / PASS; Sprint 16 COMPLETED / PASS; Sprint 17 PENDING / READY TO OPEN; H6 BLOCKED / NOT RUN

Estado postlanzamiento: NOT OPEN

Fecha de verificación de Steamworks: 1 de julio de 2026

Aviso: este documento es un plan operativo y de gobernanza. No sustituye asesoramiento jurídico, laboral, fiscal, de protección de datos, seguridad o comunicación de crisis. Los términos y herramientas de Steamworks deben revalidarse antes de cada activación material.

Resumen ejecutivo

El proyecto todavía no está publicado y no dispone de AppID, Steamworks SDK, Community Hub, Steam Cloud, telemetría, crash reporting externo, ticketing ni dirección pública de soporte. Por tanto, este plan no describe una operación existente: define las condiciones, procedimientos, plantillas y gates que deberán estar preparados antes de aceptar jugadores públicos.

La base técnica aporta ventajas importantes: guardado local atómico, backup y recuperación; estado económico determinista; baseline de QA; versionado; builds históricas; y un plan de Steam detallado. Sin embargo, la publicación sigue

bloqueada por StoreInitial, Sprint 17/H6, localización, audio, fuente, notices, clearance, soporte y autorización legal.

Área	Estado actual	Decisión
Juego público	No publicado	NOT OPEN
Community Hub	No existe	Preparar antes de Coming Soon
Soporte público	No existe	Definir email/Hub/ticketing mínimo
Telemetría	No integrada	NOT OPEN hasta plan de privacidad
Crash reporting	No integrado	Evaluar solución moderna; Steam legacy no apto para x64
Steam Cloud	No integrado	Evaluar después de migración y conflicto de saves
Save local	Atómico, backup, recovery, schema 2	Base válida, migradores pendientes
Builds	17 PASS + 3 pendientes	Ninguna pública/elegible
Rollback Steam	Documentado, no probado	Requiere Ap-pID/branches/manifests
Baseline de regresión	1215 EditMode + 70 PlayMode	Debe ampliarse con campañas postlanzamiento

Convenciones de estado

Estado	Significado
CURRENT	Evidencia real observada en proyecto o documento vigente.
PLANNED	Trabajo aprobado como plan, aún no ejecutado.
DEFERRED	Válido, pero pospuesto por prioridad o dependencia.
NOT OPEN	Sistema o compromiso no autorizado.
BLOCKED	No puede avanzar hasta cerrar una condición.
HISTORICAL	Se conserva como genealogía, no como autoridad actual.

0. Propósito, alcance y autoridad

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **propósito, alcance y autoridad** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidata pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo. 1. Definir owner principal y suplente. 2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7. 3. Crear identidad y permisos separados para soporte/moderación. 4. Publicar qué información puede enviar el jugador y qué debe ocultar. 5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

1. Genealogía histórica

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **genealogía histórica** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidata pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo. 1. Definir owner principal y suplente. 2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7. 3. Crear identidad y permisos separados para soporte/moderación. 4. Publicar

qué información puede enviar el jugador y qué debe ocultar. 5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

2. Diferencia entre soporte, live operations y desarrollo futuro

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **diferencia entre soporte, live operations y desarrollo futuro** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidata pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo. 1. Definir owner principal y suplente. 2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7. 3. Crear identidad y permisos separados para soporte/moderación. 4. Publicar qué información puede enviar el jugador y qué debe ocultar. 5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo. 6. Registrar en el ticket o release record el impacto específico de **Diferencia entre soporte, live operations y desarrollo futuro** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

3. Estado actual del proyecto

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **estado actual del proyecto** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La aplicación observada es 0.0.21, Windows x64, Unity 6000.3.18f1. Sprints 0–16 y BLD-016-POST están en PASS; Sprint 17 está PENDING / READY TO OPEN y H6 permanece BLOCKED / NOT RUN. El registro legal cuenta 0 builds elegibles para publicación, 9 riesgos High/Critical y 7 deudas S1.

Regla operativa. No se denomina postlanzamiento a una fase que todavía no tiene producto publicado, AppID, branch pública ni autorización legal.

Procedimiento mínimo. 1. Definir owner principal y suplente. 2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7. 3. Crear identidad y permisos separados para soporte/moderación. 4. Publicar qué información puede enviar el jugador y qué debe ocultar. 5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo. 6. Registrar en el ticket o release record el impacto específico de **Estado actual del proyecto** y su relación con versión, build, save y comunicación.

Evidencia mínima. Estado consolidado de producción, legal, Steam y QA con fecha y hash.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

4. Principios operativos

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **principios operativos** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidata pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo. 1. Definir owner principal y suplente. 2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7. 3. Crear identidad y permisos separados para soporte/moderación. 4. Publicar qué información puede enviar el jugador y qué debe ocultar. 5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo. 6. Registrar en el ticket o release record el impacto específico de **Principios operativos** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

5. Horario y capacidad de soporte

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **horario y capacidad de soporte** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidata pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo. 1. Definir owner principal y suplente. 2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7. 3. Crear identidad y permisos separados para soporte/moderación. 4. Publicar qué información puede enviar el jugador y qué debe ocultar. 5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo. 6. Registrar en el ticket o release record el impacto específico de **Horario y capacidad de soporte** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

6. Canales de entrada

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **canales de entrada** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidata pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo. 1. Definir owner principal y suplente. 2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7. 3. Crear identidad y permisos separados para soporte/moderación. 4. Publicar qué información puede enviar el jugador y qué debe ocultar. 5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo. 6. Registrar en el ticket o release record el impacto específico de **Canales de entrada** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

7. Email de soporte

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **email de soporte** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe una dirección pública documentada. Debe crearse una cuenta del dominio oficial solo después de cerrar identidad, dominio, privacidad y capacidad de atención.

Regla operativa. No se publica una dirección personal ni se usa como repositorio de saves indefinido. Se configura MFA, recuperación y retención.

Procedimiento mínimo. 1. Definir owner principal y suplente. 2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7. 3. Crear identidad y permisos separados para soporte/moderación. 4. Publicar qué información puede enviar el jugador y qué debe ocultar. 5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo. 6. Registrar en el ticket o release record el impacto específico de **Email de soporte** y su relación con versión, build, save y comunicación.

Evidencia mínima. Prueba de envío/recepción, autorespuesta, acceso, backup y política de datos.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

8. Steam Community Hub

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **steam community hub** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El Hub aparecerá cuando la aplicación alcance Coming Soon. Steam permite foros específicos para bugs o idiomas y muestra como desarrollador a las cuentas con permisos adecuados.

Regla operativa. Antes de Coming Soon deben existir reglas, permisos, owner y un plan de moderación. Valve revisa contenido reportado, pero el estudio mantiene responsabilidad operativa.

Procedimiento mínimo. 1. Definir owner principal y suplente. 2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7. 3. Crear identidad y permisos separados para soporte/moderación. 4. Publicar qué información puede enviar el jugador y qué debe ocultar. 5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo. 6. Registrar en el ticket o release record el impacto específico de **Steam Community Hub** y su relación con versión, build, save y comunicación.

Evidencia mínima. Captura de Hub, foros, permisos, reglas y prueba de moderación.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

Los upgrades se realizan en rama aislada, con lockfile, changelog, licencia/NOTICE, build y regresión antes de decidir adopción.

9. Foro de bugs

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **foro de bugs** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidata pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo. 1. Definir owner principal y suplente. 2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7. 3. Crear identidad y permisos separados para soporte/moderación. 4. Publicar qué información puede enviar el jugador y qué debe ocultar. 5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo. 6. Registrar en el ticket o release record el impacto específico de **Foro de bugs** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

10. Foro de sugerencias

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **foro de sugerencias** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidata pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo. 1. Definir owner principal y suplente. 2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7. 3. Crear identidad y permisos separados para soporte/moderación. 4. Publicar qué información puede enviar el jugador y qué debe ocultar. 5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo. 6. Registrar en el ticket o release record el impacto específico de **Foro de sugerencias** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

11. Reseñas

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **reseñas** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Las reviews son feedback sobre juego, valor, prácticas y comunidad. No todas requieren respuesta; una respuesta oficial puede amplificar una discusión.

Regla operativa. No solicitar reviews dentro del juego ni ofrecer recompensas. No refutar opiniones. Responder solo con claridad, corrección factual, solución

o enlace a notas.

Procedimiento mínimo. 1. Definir owner principal y suplente. 2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7. 3. Crear identidad y permisos separados para soporte/moderación. 4. Publicar qué información puede enviar el jugador y qué debe ocultar. 5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo. 6. Registrar en el ticket o release record el impacto específico de **Reseñas** y su relación con versión, build, save y comunicación.

Evidencia mínima. Registro de review, decisión de responder, wording aprobado y resultado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

La acción pública debe ser proporcional, consistente y explicable; se conserva un registro interno sin publicar datos personales del usuario.

12. Redes y web

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **redes y web** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidata pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo. 1. Definir owner principal y suplente. 2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7. 3. Crear identidad y permisos separados para soporte/moderación. 4. Publicar qué información puede enviar el jugador y qué debe ocultar. 5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo. 6. Registrar en el ticket o release record el impacto específico de **Redes y web** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

13. Privacidad

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **privacidad** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No hay telemetría, Cloud ni servicio online integrado; por ello la superficie de datos actual es principalmente local. El soporte futuro puede recibir logs y saves si el jugador los envía voluntariamente.

Regla operativa. Aplicar minimización, propósito, retención y acceso restringido. No prometer anonimato si los archivos contienen identificadores o rutas.

Procedimiento mínimo. 1. Definir owner principal y suplente. 2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7. 3. Crear identidad y permisos separados para soporte/moderación. 4. Publicar qué información puede enviar el jugador y qué debe ocultar. 5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo. 6. Registrar en el ticket o release record el impacto específico de **Privacidad** y su relación con versión, build, save y comunicación.

Evidencia mínima. Inventario de datos, notice de soporte, retención y procedimiento de borrado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

14. Datos mínimos solicitados

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **datos mínimos solicitados** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidata pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo. 1. Definir owner principal y suplente. 2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7. 3. Crear identidad y permisos separados para soporte/moderación. 4. Publicar qué información puede enviar el jugador y qué debe ocultar. 5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo. 6. Registrar en el ticket o release

record el impacto específico de **Datos mínimos solicitados** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

15. Logs

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **logs** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El artefacto primario es `Player.log`; el proyecto también conserva logs y reportes de builds internas. No hay envío automático.

Regla operativa. El log se pide con instrucciones por plataforma y se revisa para retirar nombres de usuario, rutas o información ajena al incidente.

Procedimiento mínimo. 1. Definir owner principal y suplente. 2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7. 3. Crear identidad y permisos separados para soporte/moderación. 4. Publicar qué información puede enviar el jugador y qué debe ocultar. 5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo. 6. Registrar en el ticket o release record el impacto específico de **Logs** y su relación con versión, build, save y comunicación.

Evidencia mínima. Log hashado, versión, ventana temporal y análisis asociado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

16. Saves

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **saves** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Los guardados integrados viven bajo `Application.persistentDataPath/IntegratedSaveGames` usan `primary`, `.bak`, `.tmp` y `.recovery`, generación creciente y validación. El esquema actual es 2.

Regla operativa. Un save enviado no se modifica sobre la única copia. Se duplica, se trabaja en entorno aislado y se conserva cadena de custodia.

Procedimiento mínimo. 1. Definir owner principal y suplente. 2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7. 3. Crear identidad y permisos separados para soporte/moderación. 4. Publicar qué información puede enviar el jugador y qué debe ocultar. 5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo. 6. Registrar en el ticket o release record el impacto específico de **Saves** y su relación con versión, build, save y comunicación.

Evidencia mínima. Save original hashado, copia de trabajo, resultado de validación y herramienta usada.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

Las pruebas usan fixtures reales anonimizados, primary/backup y estados límite; nunca trabajan sobre la única copia del jugador.

17. Consentimiento

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **consentimiento** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidata pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo. 1. Definir owner principal y suplente. 2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7. 3. Crear identidad y permisos separados para soporte/moderación. 4. Publicar qué información puede enviar el jugador y qué debe ocultar. 5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo. 6. Registrar en el ticket o release record el impacto específico de **Consentimiento** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

18. Clasificación de incidencias

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **clasificación de incidencias** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidata pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo. 1. Definir owner principal y suplente. 2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7. 3. Crear identidad y permisos separados para soporte/moderación. 4. Publicar qué información puede enviar el jugador y qué debe ocultar. 5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo. 6. Registrar en el ticket o release record el impacto específico de **Clasificación de incidencias** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

19. Severidades S0–S4

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **severidades s0–s4** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo. 1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción. 2. Separar incidente, defecto, consulta, suge-

rencia y abuso. 3. Buscar duplicados antes de abrir trabajo nuevo. 4. Definir workaround seguro sin alterar datos críticos. 5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas. 6. Registrar en el ticket o release record el impacto específico de **Severidades S0-S4** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

Severidad	Ejemplo postlanzamiento	Respuesta
S0	riesgo de seguridad, pérdida masiva de datos o build dañina	retirada/rollback inmediato y comunicación de crisis
S1	no arranca, corrupción reproducible, softlock general, exploit económico persistente	congelar release, hotfix/rollback prioritario
S2	feature importante rota con workaround limitado	patch prioritario y known issue
S3	defecto menor o localizado	backlog de patch
S4	cosmético, sugerencia o mejora	priorización ordinaria

20. Prioridades

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **prioridades** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo. 1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción. 2. Separar incidente, defecto, consulta, suge-

rencia y abuso. 3. Buscar duplicados antes de abrir trabajo nuevo. 4. Definir workaround seguro sin alterar datos críticos. 5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas. 6. Registrar en el ticket o release record el impacto específico de **Prioridades** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

21. Impacto

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **impacto** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo. 1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción. 2. Separar incidente, defecto, consulta, sugerencia y abuso. 3. Buscar duplicados antes de abrir trabajo nuevo. 4. Definir workaround seguro sin alterar datos críticos. 5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas. 6. Registrar en el ticket o release record el impacto específico de **Impacto** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

22. Frecuencia

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **frecuencia** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo. 1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción. 2. Separar incidente, defecto, consulta, sugerencia y abuso. 3. Buscar duplicados antes de abrir trabajo nuevo. 4. Definir workaround seguro sin alterar datos críticos. 5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas. 6. Registrar en el ticket o release record el impacto específico de **Frecuencia** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

23. Reproducibilidad

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **reproducibilidad** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo. 1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción. 2. Separar incidente, defecto, consulta, sugerencia y abuso. 3. Buscar duplicados antes de abrir trabajo nuevo. 4. Definir workaround seguro sin alterar datos críticos. 5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas. 6. Registrar en el ticket o release record el impacto específico de **Reproducibilidad** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

24. Alcance

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **alcance** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo. 1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción. 2. Separar incidente, defecto, consulta, sugerencia y abuso. 3. Buscar duplicados antes de abrir trabajo nuevo. 4. Definir workaround seguro sin alterar datos críticos. 5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas. 6. Registrar en el ticket o release record el impacto específico de **Alcance** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

25. Triage

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **triage** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo. 1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción. 2. Separar incidente, defecto, consulta, sugerencia y abuso. 3. Buscar duplicados antes de abrir trabajo nuevo. 4. Definir workaround seguro sin alterar datos críticos. 5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas. 6. Registrar en el ticket o release record el impacto específico de **Triage** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

26. Duplicados

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **duplicados** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo. 1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción. 2. Separar incidente, defecto, consulta, sugerencia y abuso. 3. Buscar duplicados antes de abrir trabajo nuevo. 4. Definir workaround seguro sin alterar datos críticos. 5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas. 6. Registrar en el ticket o release record el impacto específico de **Duplicados** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

27. Bugs conocidos

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **bugs conocidos** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo. 1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción. 2. Separar incidente, defecto, consulta, sugerencia y abuso. 3. Buscar duplicados antes de abrir trabajo nuevo. 4. Definir workaround seguro sin alterar datos críticos. 5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas. 6. Registrar en el ticket o release record el impacto específico de **Bugs conocidos** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

28. Workarounds

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **workarounds** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo. 1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción. 2. Separar incidente, defecto, consulta, sugere-

rencia y abuso. 3. Buscar duplicados antes de abrir trabajo nuevo. 4. Definir workaround seguro sin alterar datos críticos. 5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas. 6. Registrar en el ticket o release record el impacto específico de **Workarounds** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

29. Escalado

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **escalado** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo. 1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción. 2. Separar incidente, defecto, consulta, sugerencia y abuso. 3. Buscar duplicados antes de abrir trabajo nuevo. 4. Definir workaround seguro sin alterar datos críticos. 5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas. 6. Registrar en el ticket o release record el impacto específico de **Escalado** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

30. Incidentes críticos

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **incidentes críticos** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo. 1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción. 2. Separar incidente, defecto, consulta, sugerencia y abuso. 3. Buscar duplicados antes de abrir trabajo nuevo. 4. Definir workaround seguro sin alterar datos críticos. 5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas. 6. Registrar en el ticket o release record el impacto específico de **Incidentes críticos** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

31. Caídas de lanzamiento

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **caídas de lanzamiento** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo. 1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción. 2. Separar incidente, defecto, consulta, sugerencia y abuso. 3. Buscar duplicados antes de abrir trabajo nuevo. 4. Definir workaround seguro sin alterar datos críticos. 5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas. 6. Registrar en el ticket o release record el impacto específico de **Caídas de lanzamiento** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

32. Pérdida de save

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **pérdida de save** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Es un incidente potencial S0/S1 porque afecta progreso y confianza. El repositorio puede recuperar backup, pero una pérdida más allá de primary/backup necesita procedimiento manual y comunicación prioritaria.

Regla operativa. Congelar acciones que sobrescriban archivos, pedir copia completa de la carpeta y evitar instrucciones destructivas.

Procedimiento mínimo. 1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción. 2. Separar incidente, defecto, consulta, sugerencia y abuso. 3. Buscar duplicados antes de abrir trabajo nuevo. 4. Definir workaround seguro sin alterar datos críticos. 5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas. 6. Registrar en el ticket o release record el impacto específico de **Pérdida de save** y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete primary/backup/recovery, logs, versión, causa y plan de restauración.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

Las pruebas usan fixtures reales anonimizados, primary/backup y estados límite; nunca trabajan sobre la única copia del jugador.

33. Corrupción

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **corrupción** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El codec rechaza schema no soportado, generación inválida y datos inconsistentes; el repositorio intenta backup antes de declarar `CorruptPrimaryNoBackup`.

Regla operativa. No “arreglar” JSON a mano sin herramienta versionada, validación y backup. Toda reparación debe producir un nuevo archivo y registro.

Procedimiento mínimo. 1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción. 2. Separar incidente, defecto, consulta, sugerencia y abuso. 3. Buscar duplicados antes de abrir trabajo nuevo. 4. Definir workaround seguro sin alterar datos críticos. 5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas. 6. Registrar en el ticket o release record el impacto específico de **Corrupción** y su relación con versión, build, save y comunicación.

Evidencia mínima. Diagnóstico de codec, herramienta de reparación, tests y save restaurado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

Las pruebas usan fixtures reales anonimizados, primary/backup y estados límite; nunca trabajan sobre la única copia del jugador.

34. Crash

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **crash** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe SDK externo. La función histórica Steam Error Reporting no es apropiada para el objetivo Windows x64 porque la documentación oficial la marca cercana a EOL y limitada a 32 bits.

Regla operativa. Elegir explícitamente entre soporte manual, dumps del sistema o un proveedor moderno, con revisión legal y de privacidad.

Procedimiento mínimo. 1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción. 2. Separar incidente, defecto, consulta, sugerencia y abuso. 3. Buscar duplicados antes de abrir trabajo nuevo. 4. Definir workaround seguro sin alterar datos críticos. 5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas. 6. Registrar en el ticket o release record el impacto específico de **Crash** y su relación con versión, build, save y comunicación.

Evidencia mínima. Crash reproducible, log/dump, símbolos, build exacta, stack y fix validado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

35. Bloqueo de progreso

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **bloqueo de progreso** después de una publicación. La regla se

aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo. 1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción. 2. Separar incidente, defecto, consulta, sugerencia y abuso. 3. Buscar duplicados antes de abrir trabajo nuevo. 4. Definir workaround seguro sin alterar datos críticos. 5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas. 6. Registrar en el ticket o release record el impacto específico de **Bloqueo de progreso** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

Las pruebas usan fixtures reales anonimizados, primary/backup y estados límite; nunca trabajan sobre la única copia del jugador.

36. Economía rota

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **economía rota** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo. 1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción. 2. Separar incidente, defecto, consulta, sugerencia y abuso. 3. Buscar duplicados antes de abrir trabajo nuevo. 4. Definir

workaround seguro sin alterar datos críticos. 5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas. 6. Registrar en el ticket o release record el impacto específico de **Economía rota** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

37. Duplicación

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **duplicación** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo. 1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción. 2. Separar incidente, defecto, consulta, sugerencia y abuso. 3. Buscar duplicados antes de abrir trabajo nuevo. 4. Definir workaround seguro sin alterar datos críticos. 5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas. 6. Registrar en el ticket o release record el impacto específico de **Duplicación** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

38. Localización

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **localización** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo. 1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción. 2. Separar incidente, defecto, consulta, sugerencia y abuso. 3. Buscar duplicados antes de abrir trabajo nuevo. 4. Definir workaround seguro sin alterar datos críticos. 5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas. 6. Registrar en el ticket o release record el impacto específico de **Localización** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

ES y EN se publican juntas para información crítica; una traducción pendiente no se sustituye silenciosamente por texto técnico o raw IDs.

39. Audio

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **audio** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo. 1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción. 2. Separar incidente, defecto, consulta, sugerencia y abuso. 3. Buscar duplicados antes de abrir trabajo nuevo. 4. Definir workaround seguro sin alterar datos críticos. 5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas. 6. Registrar en el ticket o release

record el impacto específico de **Audio** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

El diagnóstico compara hardware, OS, drivers, resolución, branch y build; se evita atribuir el problema a la plataforma sin reproducción controlada.

40. Rendimiento

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **rendimiento** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo. 1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción. 2. Separar incidente, defecto, consulta, sugerencia y abuso. 3. Buscar duplicados antes de abrir trabajo nuevo. 4. Definir workaround seguro sin alterar datos críticos. 5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas. 6. Registrar en el ticket o release record el impacto específico de **Rendimiento** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

El diagnóstico compara hardware, OS, drivers, resolución, branch y build; se evita atribuir el problema a la plataforma sin reproducción controlada.

41. Compatibilidad

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **compatibilidad** después de una publicación. La regla se aplica tan-

to a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo. 1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción. 2. Separar incidente, defecto, consulta, sugerencia y abuso. 3. Buscar duplicados antes de abrir trabajo nuevo. 4. Definir workaround seguro sin alterar datos críticos. 5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas. 6. Registrar en el ticket o release record el impacto específico de **Compatibilidad** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

El diagnóstico compara hardware, OS, drivers, resolución, branch y build; se evita atribuir el problema a la plataforma sin reproducción controlada.

42. Instalación

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **instalación** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo. 1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción. 2. Separar incidente, defecto, consulta, sugerencia y abuso. 3. Buscar duplicados antes de abrir trabajo nuevo. 4. Definir

workaround seguro sin alterar datos críticos. 5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas. 6. Registrar en el ticket o release record el impacto específico de **Instalación** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

43. Actualización

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **actualización** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo. 1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción. 2. Separar incidente, defecto, consulta, sugerencia y abuso. 3. Buscar duplicados antes de abrir trabajo nuevo. 4. Definir workaround seguro sin alterar datos críticos. 5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas. 6. Registrar en el ticket o release record el impacto específico de **Actualización** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

44. Antivirus

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **antivirus** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo. 1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción. 2. Separar incidente, defecto, consulta, sugerencia y abuso. 3. Buscar duplicados antes de abrir trabajo nuevo. 4. Definir workaround seguro sin alterar datos críticos. 5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas. 6. Registrar en el ticket o release record el impacto específico de **Antivirus** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

El diagnóstico compara hardware, OS, drivers, resolución, branch y build; se evita atribuir el problema a la plataforma sin reproducción controlada.

45. Steam Cloud

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **steam cloud** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No está implementado. El guardado actual es local y atómico. Steam Cloud puede usar Auto-Cloud o API, sincroniza antes/después de sesión y puede probarse en modo developers-only.

Regla operativa. No activar Cloud hasta probar conflictos, backup, cuota, múltiples equipos, desactivación por usuario, suspend/resume y compatibilidad de schema.

Procedimiento mínimo. 1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción. 2. Separar incidente, defecto, consulta, sugerencia y abuso. 3. Buscar duplicados antes de abrir trabajo nuevo. 4. Definir workaround seguro sin alterar datos críticos. 5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas. 6. Registrar en el ticket o release record el impacto específico de **Steam Cloud** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de paths, cuotas, pruebas en dos equipos, `cloud_log.txt` y decisión.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

46. Proton y Steam Deck

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **proton y steam deck** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo. 1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción. 2. Separar incidente, defecto, consulta, sugerencia y abuso. 3. Buscar duplicados antes de abrir trabajo nuevo. 4. Definir workaround seguro sin alterar datos críticos. 5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas. 6. Registrar en el ticket o release record el impacto específico de **Proton y Steam Deck** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

El diagnóstico compara hardware, OS, drivers, resolución, branch y build; se evita atribuir el problema a la plataforma sin reproducción controlada.

47. Plantilla de reporte

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **plantilla de reporte** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash,

instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo. 1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción. 2. Separar incidente, defecto, consulta, sugerencia y abuso. 3. Buscar duplicados antes de abrir trabajo nuevo. 4. Definir workaround seguro sin alterar datos críticos. 5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas. 6. Registrar en el ticket o release record el impacto específico de **Plantilla de reporte** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

Título:

Versión / build / branch / manifest:

Sistema operativo y hardware:

Idioma y resolución:

Pasos exactos:

Resultado esperado:

Resultado real:

Frecuencia:

Player.log adjunto: sí/no

Save adjunto y consentimiento: sí/no

Workaround probado:

Notas:

48. Evidencia mínima

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **evidencia mínima** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de **Player.log**, builds versionadas internamente y guardados JSON con **primary**, **.bak**, **.tmp** y **.recovery**. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se

conservan con acceso limitado.

Procedimiento mínimo. 1. Proporcionar instrucciones exactas para localizar log y save. 2. Redactar datos personales antes de adjuntar evidencia. 3. Asignar ID al incidente y al artefacto recibido. 4. Calcular hash y preservar copia inmutable. 5. Eliminar o anonimizar según la política de retención. 6. Registrar en el ticket o release record el impacto específico de **Evidencia mínima** y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015.

49. Player.log

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **player.log** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de **Player.log**, builds versionadas internamente y guardados JSON con **primary**, **.bak**, **.tmp** y **.recovery**. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo. 1. Proporcionar instrucciones exactas para localizar log y save. 2. Redactar datos personales antes de adjuntar evidencia. 3. Asignar ID al incidente y al artefacto recibido. 4. Calcular hash y preservar copia inmutable. 5. Eliminar o anonimizar según la política de retención. 6. Registrar en el ticket o release record el impacto específico de **Player.log** y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015.

50. Dumps

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **dumps** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la

primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con `primary`, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo. 1. Proporcionar instrucciones exactas para localizar log y save. 2. Redactar datos personales antes de adjuntar evidencia. 3. Asignar ID al incidente y al artefacto recibido. 4. Calcular hash y preservar copia inmutable. 5. Eliminar o anonimizar según la política de retención. 6. Registrar en el ticket o release record el impacto específico de **Dumps** y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015.

51. Hardware

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **hardware** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con `primary`, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo. 1. Proporcionar instrucciones exactas para localizar log y save. 2. Redactar datos personales antes de adjuntar evidencia. 3. Asignar ID al incidente y al artefacto recibido. 4. Calcular hash y preservar copia inmutable. 5. Eliminar o anonimizar según la política de retención. 6. Registrar en el ticket o release record el impacto específico de **Hardware** y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015.

52. Sistema operativo

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **sistema operativo** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con `primary`, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo. 1. Proporcionar instrucciones exactas para localizar log y save. 2. Redactar datos personales antes de adjuntar evidencia. 3. Asignar ID al incidente y al artefacto recibido. 4. Calcular hash y preservar copia inmutable. 5. Eliminar o anonimizar según la política de retención. 6. Registrar en el ticket o release record el impacto específico de **Sistema operativo** y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015.

53. Idioma

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **idioma** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con `primary`, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real

ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo. 1. Proporcionar instrucciones exactas para localizar log y save. 2. Redactar datos personales antes de adjuntar evidencia. 3. Asignar ID al incidente y al artefacto recibido. 4. Calcular hash y preservar copia inmutable. 5. Eliminar o anonimizar según la política de retención. 6. Registrar en el ticket o release record el impacto específico de **Idioma** y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015.

ES y EN se publican juntas para información crítica; una traducción pendiente no se sustituye silenciosamente por texto técnico o raw IDs.

54. Versión

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **versión** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con `primary`, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo. 1. Proporcionar instrucciones exactas para localizar log y save. 2. Redactar datos personales antes de adjuntar evidencia. 3. Asignar ID al incidente y al artefacto recibido. 4. Calcular hash y preservar copia inmutable. 5. Eliminar o anonimizar según la política de retención. 6. Registrar en el ticket o release record el impacto específico de **Versión** y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015.

La identificación técnica se conserva como tupla `app version + commit + Steam build + depot manifest + branch`, evitando que dos artefactos distintos compartan una etiqueta ambigua.

55. Branch

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **branch** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con `primary`, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo. 1. Proporcionar instrucciones exactas para localizar log y save. 2. Redactar datos personales antes de adjuntar evidencia. 3. Asignar ID al incidente y al artefacto recibido. 4. Calcular hash y preservar copia inmutable. 5. Eliminar o anonimizar según la política de retención. 6. Registrar en el ticket o release record el impacto específico de **Branch** y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015.

La identificación técnica se conserva como tupla `app version + commit + Steam build + depot manifest + branch`, evitando que dos artefactos distintos compartan una etiqueta ambigua.

56. Manifest ID

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **manifest id** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con `primary`, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo. 1. Proporcionar instrucciones exactas para localizar log y save. 2. Redactar datos personales antes de adjuntar evidencia. 3. Asignar ID al incidente y al artefacto recibido. 4. Calcular hash y preservar copia inmutable. 5. Eliminar o anonimizar según la política de retención. 6. Registrar en el ticket o release record el impacto específico de **Manifest ID** y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015.

La identificación técnica se conserva como tupla `app version + commit + Steam build + depot manifest + branch`, evitando que dos artefactos distintos compartan una etiqueta ambigua.

57. Save afectado

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **save afectado** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con `primary`, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo. 1. Proporcionar instrucciones exactas para localizar log y save. 2. Redactar datos personales antes de adjuntar evidencia. 3. Asignar ID al incidente y al artefacto recibido. 4. Calcular hash y preservar copia inmutable. 5. Eliminar o anonimizar según la política de retención. 6. Registrar en el ticket o release record el impacto específico de **Save afectado** y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015.

Las pruebas usan fixtures reales anonimizados, `primary/backup` y estados límite; nunca trabajan sobre la única copia del jugador.

58. Reproducción

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **reproducción** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con `primary`, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo. 1. Proporcionar instrucciones exactas para localizar log y save. 2. Redactar datos personales antes de adjuntar evidencia. 3. Asignar ID al incidente y al artefacto recibido. 4. Calcular hash y preservar copia inmutable. 5. Eliminar o anonimizar según la política de retención. 6. Registrar en el ticket o release record el impacto específico de **Reproducción** y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015.

59. Registro de incidencias

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **registro de incidencias** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con `primary`, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo. 1. Proporcionar instrucciones exactas para localizar log y save. 2. Redactar datos personales antes de adjuntar evidencia. 3. Asignar

ID al incidente y al artefacto recibido. 4. Calcular hash y preservar copia inmutable. 5. Eliminar o anonimizar según la política de retención. 6. Registrar en el ticket o release record el impacto específico de **Registro de incidencias** y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015.

Campo	Obligatorio
Incident ID	sí
Fecha/hora y zona	sí
Fuente/canal	sí
Severidad/priority	sí
Build/manifest/branch	sí
Reproducción	sí para bug
Save/log/dump hash	cuando aplique
Owner	sí
Decisión y comunicación	sí
Release que corrige	al cerrar

60. SLA internos

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **sla internos** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con `primary`, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo. 1. Proporcionar instrucciones exactas para localizar log y save. 2. Redactar datos personales antes de adjuntar evidencia. 3. Asignar ID al incidente y al artefacto recibido. 4. Calcular hash y preservar copia inmutable. 5. Eliminar o anonimizar según la política de retención. 6. Registrar en el ticket o release record el impacto específico de **SLA internos** y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015.

Clase	Acuse objetivo	Triage objetivo	Decisión objetivo
S0	tan pronto como sea detectado	inmediato	retirada/contención en la misma sesión operativa
S1	mismo día laborable	mismo día	siguiente ventana razonable tras reproducción
S2	1–2 días laborables	2 días	incluir o rechazar con motivo
S3/S4	sin garantía pública	revisión periódica	backlog/closed

61. Tiempo de primera respuesta

Este capítulo forma parte del bloque **SLA y comunicación de incidente** y define cómo se gobierna **tiempo de primera respuesta** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe equipo 24/7. El plan debe ser viable para una producción individual y evitar compromisos que no puedan cumplirse. Los SLA son objetivos internos de respuesta y decisión, no garantías contractuales públicas.

Regla operativa. La primera comunicación reconoce el problema, delimita lo conocido y explica el siguiente punto de actualización. Nunca se promete una fecha de corrección sin reproducción, solución candidata y QA suficiente.

Procedimiento mínimo. 1. Confirmar recepción y severidad provisional. 2. Comunicar workaround solo si es seguro. 3. Fijar el siguiente update por condición o ventana realista. 4. Separar investigación, solución, validación y publicación. 5. Cerrar con versión corregida, notas y evidencia. 6. Registrar en el ticket o release record el impacto específico de **Tiempo de primera respuesta** y su relación con versión, build, save y comunicación.

Evidencia mínima. Cronología del incidente, mensajes publicados, tiempos internos y criterio de cierre.

Fuentes Steamworks relacionadas: ST-OPS-002; ST-OPS-003; ST-OPS-010.

62. Tiempo de triage

Este capítulo forma parte del bloque **SLA y comunicación de incidente** y define cómo se gobierna **tiempo de triage** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe equipo 24/7. El plan debe ser viable para una producción individual y evitar compromisos que no puedan cumplirse. Los SLA son objetivos internos de respuesta y decisión, no garantías contractuales públicas.

Regla operativa. La primera comunicación reconoce el problema, delimita lo conocido y explica el siguiente punto de actualización. Nunca se promete una fecha de corrección sin reproducción, solución candidata y QA suficiente.

Procedimiento mínimo. 1. Confirmar recepción y severidad provisional. 2. Comunicar workaround solo si es seguro. 3. Fijar el siguiente update por condición o ventana realista. 4. Separar investigación, solución, validación y publicación. 5. Cerrar con versión corregida, notas y evidencia. 6. Registrar en el ticket o release record el impacto específico de **Tiempo de triage** y su relación con versión, build, save y comunicación.

Evidencia mínima. Cronología del incidente, mensajes publicados, tiempos internos y criterio de cierre.

Fuentes Steamworks relacionadas: ST-OPS-002; ST-OPS-003; ST-OPS-010.

63. Tiempo de decisión

Este capítulo forma parte del bloque **SLA y comunicación de incidente** y define cómo se gobierna **tiempo de decisión** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe equipo 24/7. El plan debe ser viable para una producción individual y evitar compromisos que no puedan cumplirse. Los SLA son objetivos internos de respuesta y decisión, no garantías contractuales públicas.

Regla operativa. La primera comunicación reconoce el problema, delimita lo conocido y explica el siguiente punto de actualización. Nunca se promete una fecha de corrección sin reproducción, solución candidata y QA suficiente.

Procedimiento mínimo. 1. Confirmar recepción y severidad provisional. 2. Comunicar workaround solo si es seguro. 3. Fijar el siguiente update por condición o ventana realista. 4. Separar investigación, solución, validación y publicación. 5. Cerrar con versión corregida, notas y evidencia. 6. Registrar en el ticket o release record el impacto específico de **Tiempo de decisión** y su relación con versión, build, save y comunicación.

Evidencia mínima. Cronología del incidente, mensajes publicados, tiempos internos y criterio de cierre.

Fuentes Steamworks relacionadas: ST-OPS-002; ST-OPS-003; ST-OPS-010.

64. Comunicación inicial

Este capítulo forma parte del bloque **SLA y comunicación de incidente** y define cómo se gobierna **comunicación inicial** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe equipo 24/7. El plan debe ser viable para una producción individual y evitar compromisos que no puedan cumplirse. Los SLA son objetivos internos de respuesta y decisión, no garantías contractuales públicas.

Regla operativa. La primera comunicación reconoce el problema, delimita lo conocido y explica el siguiente punto de actualización. Nunca se promete una fecha de corrección sin reproducción, solución candidata y QA suficiente.

Procedimiento mínimo. 1. Confirmar recepción y severidad provisional. 2. Comunicar workaround solo si es seguro. 3. Fijar el siguiente update por condición o ventana realista. 4. Separar investigación, solución, validación y publicación. 5. Cerrar con versión corregida, notas y evidencia. 6. Registrar en el ticket o release record el impacto específico de **Comunicación inicial** y su relación con versión, build, save y comunicación.

Evidencia mínima. Cronología del incidente, mensajes publicados, tiempos internos y criterio de cierre.

Fuentes Steamworks relacionadas: ST-OPS-002; ST-OPS-003; ST-OPS-010.

65. Actualizaciones de estado

Este capítulo forma parte del bloque **SLA y comunicación de incidente** y define cómo se gobierna **actualizaciones de estado** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe equipo 24/7. El plan debe ser viable para una producción individual y evitar compromisos que no puedan cumplirse. Los SLA son objetivos internos de respuesta y decisión, no garantías contractuales públicas.

Regla operativa. La primera comunicación reconoce el problema, delimita lo conocido y explica el siguiente punto de actualización. Nunca se promete una fecha de corrección sin reproducción, solución candidata y QA suficiente.

Procedimiento mínimo. 1. Confirmar recepción y severidad provisional. 2. Comunicar workaround solo si es seguro. 3. Fijar el siguiente update por condición

o ventana realista. 4. Separar investigación, solución, validación y publicación. 5. Cerrar con versión corregida, notas y evidencia. 6. Registrar en el ticket o release record el impacto específico de **Actualizaciones de estado** y su relación con versión, build, save y comunicación.

Evidencia mínima. Cronología del incidente, mensajes publicados, tiempos internos y criterio de cierre.

Fuentes Steamworks relacionadas: ST-OPS-002; ST-OPS-003; ST-OPS-010.

66. Resolución

Este capítulo forma parte del bloque **SLA y comunicación de incidente** y define cómo se gobierna **resolución** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe equipo 24/7. El plan debe ser viable para una producción individual y evitar compromisos que no puedan cumplirse. Los SLA son objetivos internos de respuesta y decisión, no garantías contractuales públicas.

Regla operativa. La primera comunicación reconoce el problema, delimita lo conocido y explica el siguiente punto de actualización. Nunca se promete una fecha de corrección sin reproducción, solución candidata y QA suficiente.

Procedimiento mínimo. 1. Confirmar recepción y severidad provisional. 2. Comunicar workaround solo si es seguro. 3. Fijar el siguiente update por condición o ventana realista. 4. Separar investigación, solución, validación y publicación. 5. Cerrar con versión corregida, notas y evidencia. 6. Registrar en el ticket o release record el impacto específico de **Resolución** y su relación con versión, build, save y comunicación.

Evidencia mínima. Cronología del incidente, mensajes publicados, tiempos internos y criterio de cierre.

Fuentes Steamworks relacionadas: ST-OPS-002; ST-OPS-003; ST-OPS-010.

67. Cierre

Este capítulo forma parte del bloque **SLA y comunicación de incidente** y define cómo se gobierna **cierre** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe equipo 24/7. El plan debe ser viable para una producción individual y evitar compromisos que no puedan cumplirse. Los SLA son objetivos internos de respuesta y decisión, no garantías contractuales públicas.

Regla operativa. La primera comunicación reconoce el problema, delimita lo conocido y explica el siguiente punto de actualización. Nunca se promete una fecha de corrección sin reproducción, solución candidata y QA suficiente.

Procedimiento mínimo. 1. Confirmar recepción y severidad provisional. 2. Comunicar workaround solo si es seguro. 3. Fijar el siguiente update por condición o ventana realista. 4. Separar investigación, solución, validación y publicación. 5. Cerrar con versión corregida, notas y evidencia. 6. Registrar en el ticket o release record el impacto específico de **Cierre** y su relación con versión, build, save y comunicación.

Evidencia mínima. Cronología del incidente, mensajes publicados, tiempos internos y criterio de cierre.

Fuentes Steamworks relacionadas: ST-OPS-002; ST-OPS-003; ST-OPS-010.

68. Hotfix

Este capítulo forma parte del bloque **SLA y comunicación de incidente** y define cómo se gobierna **hotfix** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Un hotfix corrige un defecto urgente con alcance mínimo. No incorpora features ni refactors no esenciales.

Regla operativa. Requiere reproducción, test de regresión dirigido, save/load, instalación/actualización y rollback. Incrementa PATCH cuando es público.

Procedimiento mínimo. 1. Confirmar recepción y severidad provisional. 2. Comunicar workaround solo si es seguro. 3. Fijar el siguiente update por condición o ventana realista. 4. Separar investigación, solución, validación y publicación. 5. Cerrar con versión corregida, notas y evidencia. 6. Registrar en el ticket o release record el impacto específico de **Hotfix** y su relación con versión, build, save y comunicación.

Evidencia mínima. Branch hotfix, diff, pruebas, manifest, notas y autorización.

Fuentes Steamworks relacionadas: ST-OPS-002; ST-OPS-003; ST-OPS-010.

69. Patch

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **patch** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y BLD-016-PRE en PASS; BLD-016-POST, BLD-017-QA y BLD-H6 pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo. 1. Crear rama y build desde commit limpio. 2. Incrementar versión conforme a MAJOR.MINOR.PATCH. 3. Generar checksum, manifest y release record. 4. Ejecutar reproducción, vecinos, save/load e instalación/actualización. 5. Promover manualmente tras Go/No-Go y conservar rollback. 6. Registrar en el ticket o release record el impacto específico de Patch y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

70. Update

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **update** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y BLD-016-PRE en PASS; BLD-016-POST, BLD-017-QA y BLD-H6 pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo. 1. Crear rama y build desde commit limpio. 2. Incrementar versión conforme a MAJOR.MINOR.PATCH. 3. Generar checksum, manifest y release record. 4. Ejecutar reproducción, vecinos, save/load e instalación/actualización. 5. Promover manualmente tras Go/No-Go y conservar rollback. 6. Registrar en el ticket o release record el impacto específico de Update y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

71. Content update

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **content update** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y BLD-016-PRE en PASS; BLD-016-POST, BLD-017-QA y BLD-H6 pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo. 1. Crear rama y build desde commit limpio. 2. Incrementar versión conforme a MAJOR.MINOR.PATCH. 3. Generar checksum, manifest y release record. 4. Ejecutar reproducción, vecinos, save/load e instalación/actualización. 5. Promover manualmente tras Go/No-Go y conservar rollback. 6. Registrar en el ticket o release record el impacto específico de **Content update** y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

72. Save migration

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **save migration** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El sistema actual solo acepta `CurrentSchemaVersion` = 2; no existe todavía una cadena de migradores históricos para una audiencia pública.

Regla operativa. Antes de cambiar schema se implementa migración versionada, idempotente, testeada desde cada versión soportada y con copia previa.

Procedimiento mínimo. 1. Crear rama y build desde commit limpio. 2. Incrementar versión conforme a MAJOR.MINOR.PATCH. 3. Generar checksum, manifest y release record. 4. Ejecutar reproducción, vecinos, save/load e instala-

ción/actualización. 5. Promover manualmente tras Go/No-Go y conservar roll-back. 6. Registrar en el ticket o release record el impacto específico de **Save migration** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de schemas, migradores, fixtures reales, roll-back/fallback y estadísticas de resultado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

Las pruebas usan fixtures reales anonimizados, primary/backup y estados límite; nunca trabajan sobre la única copia del jugador.

```
load original -> validate source schema -> create immutable backup
-> migrate one version at a time -> validate invariants
-> write new temporary file -> reload and compare
-> commit atomically -> preserve recovery evidence
```

73. Backward compatibility

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **backward compatibility** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y BLD-016-PRE en PASS; BLD-016-POST, BLD-017-QA y BLD-H6 pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva **previous_stable**, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo. 1. Crear rama y build desde commit limpio. 2. Incrementar versión conforme a MAJOR.MINOR.PATCH. 3. Generar checksum, manifest y release record. 4. Ejecutar reproducción, vecinos, save/load e instalación/actualización. 5. Promover manualmente tras Go/No-Go y conservar roll-back. 6. Registrar en el ticket o release record el impacto específico de **Backward compatibility** y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

74. Forward compatibility

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **forward compatibility** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y BLD-016-PRE en PASS; BLD-016-POST, BLD-017-QA y BLD-H6 pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo. 1. Crear rama y build desde commit limpio. 2. Incrementar versión conforme a MAJOR.MINOR.PATCH. 3. Generar checksum, manifest y release record. 4. Ejecutar reproducción, vecinos, save/load e instalación/actualización. 5. Promover manualmente tras Go/No-Go y conservar rollback. 6. Registrar en el ticket o release record el impacto específico de **Forward compatibility** y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

75. Versionado

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **versionado** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y BLD-016-PRE en PASS; BLD-016-POST, BLD-017-QA y BLD-H6 pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo. 1. Crear rama y build desde commit limpio. 2. Incrementar versión conforme a MAJOR.MINOR.PATCH. 3. Generar checksum, manifest y release record. 4. Ejecutar reproducción, vecinos, save/load e instalación/actualización. 5. Promover manualmente tras Go/No-Go y conservar

rollback. 6. Registrar en el ticket o release record el impacto específico de **Versionado** y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

76. Branches

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **branches** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y BLD-016-PRE en PASS; BLD-016-POST, BLD-017-QA y BLD-H6 pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva **previous_stable**, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo. 1. Crear rama y build desde commit limpio. 2. Incrementar versión conforme a **MAJOR.MINOR.PATCH**. 3. Generar checksum, manifest y release record. 4. Ejecutar reproducción, vecinos, save/load e instalación/actualización. 5. Promover manualmente tras Go/No-Go y conservar rollback. 6. Registrar en el ticket o release record el impacto específico de **Branches** y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

La identificación técnica se conserva como tupla **app version + commit + Steam build + depot manifest + branch**, evitando que dos artefactos distintos compartan una etiqueta ambigua.

77. Release candidate

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **release candidate** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y BLD-016-PRE en PASS; BLD-016-POST, BLD-017-QA y BLD-H6 pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo. 1. Crear rama y build desde commit limpio. 2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`. 3. Generar checksum, manifest y release record. 4. Ejecutar reproducción, vecinos, save/load e instalación/actualización. 5. Promover manualmente tras Go/No-Go y conservar rollback. 6. Registrar en el ticket o release record el impacto específico de `Release candidate` y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

78. Previous stable

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **previous stable** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La rama conserva la última build pública demostrada, no simplemente la penúltima subida.

Regla operativa. No se sobrescribe ni se elimina hasta que la nueva versión supere la ventana de observación y compatibilidad.

Procedimiento mínimo. 1. Crear rama y build desde commit limpio. 2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`. 3. Generar checksum, manifest y release record. 4. Ejecutar reproducción, vecinos, save/load e instalación/actualización. 5. Promover manualmente tras Go/No-Go y conservar rollback. 6. Registrar en el ticket o release record el impacto específico de `Previous stable` y su relación con versión, build, save y comunicación.

Evidencia mínima. Manifest, checksum, fecha, notas, saves de referencia y owner.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

79. Rollback

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **rollback** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Steam permite asignar builds anteriores, pero el rollback de binarios puede ser inseguro si el save ya migró.

Regla operativa. La decisión considera código, contenido, configuración y datos. Si no existe downgrade seguro, se prefiere hotfix forward o herramienta de recuperación.

Procedimiento mínimo. 1. Crear rama y build desde commit limpio. 2. Incrementar versión conforme a MAJOR.MINOR.PATCH. 3. Generar checksum, manifest y release record. 4. Ejecutar reproducción, vecinos, save/load e instalación/actualización. 5. Promover manualmente tras Go/No-Go y conservar rollback. 6. Registrar en el ticket o release record el impacto específico de Rollback y su relación con versión, build, save y comunicación.

Evidencia mínima. Manifest anterior, compatibilidad de save, prueba de rollback y comunicación.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

80. Kill switch o feature flag

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **kill switch o feature flag** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y BLD-016-PRE en PASS; BLD-016-POST, BLD-017-QA y BLD-H6 pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo. 1. Crear rama y build desde commit limpio. 2. Incrementar versión conforme a MAJOR.MINOR.PATCH. 3. Generar checksum, manifest y release record. 4. Ejecutar reproducción, vecinos, save/load e instalación/actualización. 5. Promover manualmente tras Go/No-Go y conservar rollback. 6. Registrar en el ticket o release record el impacto específico de Kill switch o feature flag y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

81. Build reproducible

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **build reproducible** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y BLD-016-PRE en PASS; BLD-016-POST, BLD-017-QA y BLD-H6 pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo. 1. Crear rama y build desde commit limpio. 2. Incrementar versión conforme a MAJOR.MINOR.PATCH. 3. Generar checksum, manifest y release record. 4. Ejecutar reproducción, vecinos, save/load e instalación/actualización. 5. Promover manualmente tras Go/No-Go y conservar rollback. 6. Registrar en el ticket o release record el impacto específico de Build reproducible y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

La identificación técnica se conserva como tupla `app version + commit + Steam build + depot manifest + branch`, evitando que dos artefactos distintos compartan una etiqueta ambigua.

82. Checksums

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **checksums** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y BLD-016-PRE en PASS; BLD-016-POST, BLD-017-QA y BLD-H6 pendientes.

SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo. 1. Crear rama y build desde commit limpio. 2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`. 3. Generar checksum, manifest y release record. 4. Ejecutar reproducción, vecinos, save/load e instalación/actualización. 5. Promover manualmente tras Go/No-Go y conservar rollback. 6. Registrar en el ticket o release record el impacto específico de **Checksums** y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

La identificación técnica se conserva como tupla `app version + commit + Steam build + depot manifest + branch`, evitando que dos artefactos distintos compartan una etiqueta ambigua.

83. QA mínima de hotfix

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **qa mínima de hotfix** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y BLD-016-PRE en PASS; BLD-016-POST, BLD-017-QA y BLD-H6 pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo. 1. Crear rama y build desde commit limpio. 2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`. 3. Generar checksum, manifest y release record. 4. Ejecutar reproducción, vecinos, save/load e instalación/actualización. 5. Promover manualmente tras Go/No-Go y conservar rollback. 6. Registrar en el ticket o release record el impacto específico de **QA mínima de hotfix** y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

84. QA completa de patch

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **qa completa de patch** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y BLD-016-PRE en PASS; BLD-016-POST, BLD-017-QA y BLD-H6 pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo. 1. Crear rama y build desde commit limpio. 2. Incrementar versión conforme a MAJOR.MINOR.PATCH. 3. Generar checksum, manifest y release record. 4. Ejecutar reproducción, vecinos, save/load e instalación/actualización. 5. Promover manualmente tras Go/No-Go y conservar rollback. 6. Registrar en el ticket o release record el impacto específico de QA completa de patch y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

85. Golden Path

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **golden path** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y BLD-016-PRE en PASS; BLD-016-POST, BLD-017-QA y BLD-H6 pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo. 1. Crear rama y build desde commit limpio. 2. Incrementar versión conforme a **MAJOR.MINOR.PATCH**. 3. Generar checksum, manifest y release record. 4. Ejecutar reproducción, vecinos, save/load e instalación/actualización. 5. Promover manualmente tras Go/No-Go y conservar rollback. 6. Registrar en el ticket o release record el impacto específico de **Golden Path** y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

86. Campaña de siete días

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **campaña de siete días** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y BLD-016-PRE en PASS; BLD-016-POST, BLD-017-QA y BLD-H6 pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva **previous_stable**, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo. 1. Crear rama y build desde commit limpio. 2. Incrementar versión conforme a **MAJOR.MINOR.PATCH**. 3. Generar checksum, manifest y release record. 4. Ejecutar reproducción, vecinos, save/load e instalación/actualización. 5. Promover manualmente tras Go/No-Go y conservar rollback. 6. Registrar en el ticket o release record el impacto específico de **Campaña de siete días** y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

87. Save/load

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **save/load** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y BLD-016-PRE en PASS; BLD-016-POST, BLD-017-QA y BLD-H6 pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo. 1. Crear rama y build desde commit limpio. 2. Incrementar versión conforme a MAJOR.MINOR.PATCH. 3. Generar checksum, manifest y release record. 4. Ejecutar reproducción, vecinos, save/load e instalación/actualización. 5. Promover manualmente tras Go/No-Go y conservar rollback. 6. Registrar en el ticket o release record el impacto específico de Save/load y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

Las pruebas usan fixtures reales anonimizados, primary/backup y estados límite; nunca trabajan sobre la única copia del jugador.

88. Instalación limpia

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **instalación limpia** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y BLD-016-PRE en PASS; BLD-016-POST, BLD-017-QA y BLD-H6 pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo. 1. Crear rama y build desde commit limpio. 2. Incrementar versión conforme a MAJOR.MINOR.PATCH. 3. Generar checksum, manifest y release record. 4. Ejecutar reproducción, vecinos, save/load e instalación/actualización. 5. Promover manualmente tras Go/No-Go y conservar rollback. 6. Registrar en el ticket o release record el impacto específico de **Instalación limpia** y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

89. Actualización desde versión anterior

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **actualización desde versión anterior** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y BLD-016-PRE en PASS; BLD-016-POST, BLD-017-QA y BLD-H6 pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo. 1. Crear rama y build desde commit limpio. 2. Incrementar versión conforme a MAJOR.MINOR.PATCH. 3. Generar checksum, manifest y release record. 4. Ejecutar reproducción, vecinos, save/load e instalación/actualización. 5. Promover manualmente tras Go/No-Go y conservar rollback. 6. Registrar en el ticket o release record el impacto específico de **Actualización desde versión anterior** y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

La identificación técnica se conserva como tupla `app version + commit + Steam build + depot manifest + branch`, evitando que dos artefactos distintos compartan una etiqueta ambigua.

90. Rollback de versión

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **rollback de versión** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y BLD-016-PRE en PASS; BLD-016-POST, BLD-017-QA y BLD-H6 pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo. 1. Crear rama y build desde commit limpio. 2. Incrementar versión conforme a MAJOR.MINOR.PATCH. 3. Generar checksum, manifest y release record. 4. Ejecutar reproducción, vecinos, save/load e instalación/actualización. 5. Promover manualmente tras Go/No-Go y conservar rollback. 6. Registrar en el ticket o release record el impacto específico de **Rollback de versión** y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

La identificación técnica se conserva como tupla `app version + commit + Steam build + depot manifest + branch`, evitando que dos artefactos distintos compartan una etiqueta ambigua.

91. Rollback de datos

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **rollback de datos** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y BLD-016-PRE en PASS; BLD-016-POST, BLD-017-QA y BLD-H6 pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo. 1. Crear rama y build desde commit limpio. 2. Incrementar versión conforme a MAJOR.MINOR.PATCH. 3. Generar checksum, manifest y release record. 4. Ejecutar reproducción, vecinos, save/load e instalación/actualización. 5. Promover manualmente tras Go/No-Go y conservar rollback. 6. Registrar en el ticket o release record el impacto específico de **Rollback de datos** y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

Las pruebas usan fixtures reales anonimizados, primary/backup y estados límite; nunca trabajan sobre la única copia del jugador.

92. SteamPipe

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **steampipe** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y BLD-016-PRE en PASS; BLD-016-POST, BLD-017-QA y BLD-H6 pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva **previous_stable**, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo. 1. Crear rama y build desde commit limpio. 2. Incrementar versión conforme a **MAJOR.MINOR.PATCH**. 3. Generar checksum, manifest y release record. 4. Ejecutar reproducción, vecinos, save/load e instalación/actualización. 5. Promover manualmente tras Go/No-Go y conservar rollback. 6. Registrar en el ticket o release record el impacto específico de **SteamPipe** y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

La identificación técnica se conserva como tupla **app version + commit + Steam build + depot manifest + branch**, evitando que dos artefactos distintos compartan una etiqueta ambigua.

93. Depots

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **depots** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y BLD-016-PRE en PASS; BLD-016-POST, BLD-017-QA y BLD-H6 pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo. 1. Crear rama y build desde commit limpio. 2. Incrementar versión conforme a MAJOR.MINOR.PATCH. 3. Generar checksum, manifest y release record. 4. Ejecutar reproducción, vecinos, save/load e instalación/actualización. 5. Promover manualmente tras Go/No-Go y conservar rollback. 6. Registrar en el ticket o release record el impacto específico de Depots y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

La identificación técnica se conserva como tupla `app version + commit + Steam build + depot manifest + branch`, evitando que dos artefactos distintos compartan una etiqueta ambigua.

94. Manifest promotion

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **manifest promotion** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y BLD-016-PRE en PASS; BLD-016-POST, BLD-017-QA y BLD-H6 pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo. 1. Crear rama y build desde commit limpio. 2. Incrementar versión conforme a MAJOR.MINOR.PATCH. 3. Generar checksum, manifest y release record. 4. Ejecutar reproducción, vecinos, save/load e instalación/actualización. 5. Promover manualmente tras Go/No-Go y conservar rollback. 6. Registrar en el ticket o release record el impacto específico de Manifest promotion y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

La identificación técnica se conserva como tupla `app version + commit + Steam build + depot manifest + branch`, evitando que dos artefactos distintos compartan una etiqueta ambigua.

95. Notas de parche

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **notas de parche** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Steam ofrece un tipo específico de Patch Notes; cada build visible debe explicar magnitud y cambios relevantes.

Regla operativa. No ocultar cambios de save, known issues o riesgos. No inflar un hotfix como gran actualización.

Procedimiento mínimo. 1. Crear rama y build desde commit limpio. 2. Incrementar versión conforme a MAJOR.MINOR.PATCH. 3. Generar checksum, manifest y release record. 4. Ejecutar reproducción, vecinos, save/load e instalación/actualización. 5. Promover manualmente tras Go/No-Go y conservar rollback. 6. Registrar en el ticket o release record el impacto específico de **Notas de parche** y su relación con versión, build, save y comunicación.

Evidencia mínima. Notas ES/EN vinculadas a versión, manifest y ticket.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

ES y EN se publican juntas para información crítica; una traducción pendiente no se sustituye silenciosamente por texto técnico o raw IDs.

96. Idiomas de las notas

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **idiomas de las notas** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y BLD-016-PRE en PASS; BLD-016-POST, BLD-017-QA y BLD-H6 pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo. 1. Crear rama y build desde commit limpio. 2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`. 3. Generar checksum, manifest y release record. 4. Ejecutar reproducción, vecinos, save/load e instalación/actualización. 5. Promover manualmente tras Go/No-Go y conservar rollback. 6. Registrar en el ticket o release record el impacto específico de **Idiomas de las notas** y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

ES y EN se publican juntas para información crítica; una traducción pendiente no se sustituye silenciosamente por texto técnico o raw IDs.

97. Comunicación de riesgos

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **comunicación de riesgos** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte solución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **Comunicación de riesgos** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

98. Known issues

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **known issues** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte solución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **Known issues** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

99. Comunidad

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **comunidad** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte solución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **Comunidad** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

La acción pública debe ser proporcional, consistente y explicable; se conserva un registro interno sin publicar datos personales del usuario.

100. Moderación

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **moderación** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Valve revisa contenido reportado, pero el proyecto debe vigilar el Hub y puede usar moderadores con permisos.

Regla operativa. Moderación por conducta y reglas publicadas. No borrar crítica legítima ni usar flags para desacuerdo.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte solución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **Moderación** y su relación con versión, build, save y comunicación.

Evidencia mínima. Log de moderación, regla aplicada, actor, fecha y posible apelación.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

La acción pública debe ser proporcional, consistente y explicable; se conserva un registro interno sin publicar datos personales del usuario.

101. Código de conducta

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **código de conducta** después de una publica-

ción. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte solución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **Código de conducta** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

102. Spam

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **spam** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte solución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **Spam** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

La acción pública debe ser proporcional, consistente y explicable; se conserva un registro interno sin publicar datos personales del usuario.

103. Abuso

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **abuso** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte solución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **Abuso** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

La acción pública debe ser proporcional, consistente y explicable; se conserva un registro interno sin publicar datos personales del usuario.

104. Contenido ilegal

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **contenido ilegal** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte solución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **Contenido ilegal** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

105. Spoilers

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **spoilers** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte solución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **Spoilers** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

La acción pública debe ser proporcional, consistente y explicable; se conserva un registro interno sin publicar datos personales del usuario.

106. Solicitudes de reembolso

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **solicitudes de reembolso** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El estudio no decide reembolsos individuales de Steam. Puede consultar informes agregados y motivos en los reportes de Steamworks.

Regla operativa. Responder con información de soporte, no presionar al jugador ni prometer resultado del reembolso. Usar tendencias para corregir expectativas o fallos.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte solución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **Solicitudes de reembolso** y su relación con versión, build, save y comunicación.

Evidencia mínima. Informe agregado, motivos categorizados y acción de producto.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

107. Comentarios negativos

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **comentarios negativos** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o

features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte solución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **Comentarios negativos** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

La acción pública debe ser proporcional, consistente y explicable; se conserva un registro interno sin publicar datos personales del usuario.

108. Reseñas negativas

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **reseñas negativas** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Una review negativa puede reflejar bugs, valor, marketing o experiencia comunitaria. Steam recomienda no responder a todas ni discutir opiniones.

Regla operativa. Responder solo cuando exista información útil, una corrección publicada o un malentendido factual importante.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte solución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **Reseñas negativas** y su relación con versión, build, save y comunicación.

Evidencia mínima. Decisión de respuesta y texto breve revisado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

La acción pública debe ser proporcional, consistente y explicable; se conserva un registro interno sin publicar datos personales del usuario.

109. Reseñas positivas

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **reseñas positivas** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte solución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **Reseñas positivas** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

La acción pública debe ser proporcional, consistente y explicable; se conserva un registro interno sin publicar datos personales del usuario.

110. Respuestas públicas

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **respuestas públicas** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte so-

lución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **Respuestas públicas** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

111. Transparencia

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **transparencia** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte solución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **Transparencia** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

112. No prometer fechas sin aprobación

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **no prometer fechas sin aprobación** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte solución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **No prometer fechas sin aprobación** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

113. Roadmap público

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **roadmap público** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte solución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **Roadmap público** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

114. Roadmap interno

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **roadmap interno** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte solución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **Roadmap interno** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

115. Sugerencias

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **sugerencias** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte solución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **Sugerencias** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

116. Votaciones

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **votaciones** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte solución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **Votaciones** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

117. Alcance futuro

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **alcance futuro** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte solución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **Alcance futuro** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

118. DLC

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **dlc** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte solución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **DLC** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

119. Expansiones

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **expansiones** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte solución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **Expansiones** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

120. Demo

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **demo** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte solución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **Demo** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

121. Playtest

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **playtest** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte solución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **Playtest** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

122. Achievements

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **achievements** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte solución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **Achievements** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

123. Cloud

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **cloud** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte solución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **Cloud** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

124. Localización futura

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **localización futura** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte solución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **Localización futura** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

ES y EN se publican juntas para información crítica; una traducción pendiente no se sustituye silenciosamente por texto técnico o raw IDs.

125. Nuevas plataformas

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **nuevas plataformas** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o

features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte solución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **Nuevas plataformas** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

126. Métricas de lanzamiento

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **métricas de lanzamiento** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo. 1. Crear foros mínimos y reglas visibles. 2. Separar bugs, sugerencias y soporte. 3. Responder públicamente solo cuando aporte solución o aclaración factual. 4. Escalar abuso mediante herramientas de Steam. 5. Revisar cada anuncio contra la build y el alcance aprobado. 6. Registrar en el ticket o release record el impacto específico de **Métricas de lanzamiento** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

127. Crash-free sessions

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **crash-free sessions** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo. 1. Definir diccionario de métricas. 2. Priorizar datos de plataforma y tickets antes de añadir SDK. 3. No incluir identificadores personales en informes ordinarios. 4. Revisar sesgos y muestras pequeñas. 5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado. 6. Registrar en el ticket o release record el impacto específico de **Crash-free sessions** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

128. Retención

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **retención** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo. 1. Definir diccionario de métricas. 2. Priorizar datos de plataforma y tickets antes de añadir SDK. 3. No incluir identificadores personales en informes ordinarios. 4. Revisar sesgos y muestras pequeñas. 5. Cerrar

el ciclo: observación, hipótesis, cambio, QA y resultado. 6. Registrar en el ticket o release record el impacto específico de **Retención** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

129. Tiempo de juego

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **tiempo de juego** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo. 1. Definir diccionario de métricas. 2. Priorizar datos de plataforma y tickets antes de añadir SDK. 3. No incluir identificadores personales en informes ordinarios. 4. Revisar sesgos y muestras pequeñas. 5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado. 6. Registrar en el ticket o release record el impacto específico de **Tiempo de juego** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

130. Abandono

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **abandono** después de una publicación. La regla se

aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo. 1. Definir diccionario de métricas. 2. Priorizar datos de plataforma y tickets antes de añadir SDK. 3. No incluir identificadores personales en informes ordinarios. 4. Revisar sesgos y muestras pequeñas. 5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado. 6. Registrar en el ticket o release record el impacto específico de **Abandono** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

131. Progreso

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **progreso** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo. 1. Definir diccionario de métricas. 2. Priorizar datos de plataforma y tickets antes de añadir SDK. 3. No incluir identificadores personales en informes ordinarios. 4. Revisar sesgos y muestras pequeñas. 5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado. 6. Registrar en el ticket o

release record el impacto específico de **Progreso** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

Las pruebas usan fixtures reales anonimizados, primary/backup y estados límite; nunca trabajan sobre la única copia del jugador.

132. Economía

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **economía** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo. 1. Definir diccionario de métricas. 2. Priorizar datos de plataforma y tickets antes de añadir SDK. 3. No incluir identificadores personales en informes ordinarios. 4. Revisar sesgos y muestras pequeñas. 5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado. 6. Registrar en el ticket o release record el impacto específico de **Economía** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

133. Stockouts

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **stockouts** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual,

QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo. 1. Definir diccionario de métricas. 2. Priorizar datos de plataforma y tickets antes de añadir SDK. 3. No incluir identificadores personales en informes ordinarios. 4. Revisar sesgos y muestras pequeñas. 5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado. 6. Registrar en el ticket o release record el impacto específico de **Stockouts** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

134. Softlocks

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **softlocks** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo. 1. Definir diccionario de métricas. 2. Priorizar datos de plataforma y tickets antes de añadir SDK. 3. No incluir identificadores personales en informes ordinarios. 4. Revisar sesgos y muestras pequeñas. 5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado. 6. Registrar en el ticket o release record el impacto específico de **Softlocks** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

135. Rendimiento operativo

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **rendimiento operativo** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo. 1. Definir diccionario de métricas. 2. Priorizar datos de plataforma y tickets antes de añadir SDK. 3. No incluir identificadores personales en informes ordinarios. 4. Revisar sesgos y muestras pequeñas. 5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado. 6. Registrar en el ticket o release record el impacto específico de **Rendimiento operativo** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

El diagnóstico compara hardware, OS, drivers, resolución, branch y build; se evita atribuir el problema a la plataforma sin reproducción controlada.

136. Soporte

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **soporte** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo. 1. Definir diccionario de métricas. 2. Priorizar datos de plataforma y tickets antes de añadir SDK. 3. No incluir identificadores personales en informes ordinarios. 4. Revisar sesgos y muestras pequeñas. 5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado. 6. Registrar en el ticket o release record el impacto específico de **Soporte** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

137. Volumen de tickets

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **volumen de tickets** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo. 1. Definir diccionario de métricas. 2. Priorizar datos de plataforma y tickets antes de añadir SDK. 3. No incluir identificadores personales en informes ordinarios. 4. Revisar sesgos y muestras pequeñas. 5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado. 6. Registrar en el ticket o release record el impacto específico de **Volumen de tickets** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

138. Tiempo de respuesta

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **tiempo de respuesta** después de una publicación. La

regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo. 1. Definir diccionario de métricas. 2. Priorizar datos de plataforma y tickets antes de añadir SDK. 3. No incluir identificadores personales en informes ordinarios. 4. Revisar sesgos y muestras pequeñas. 5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado. 6. Registrar en el ticket o release record el impacto específico de **Tiempo de respuesta** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

139. Sentimiento

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **sentimiento** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo. 1. Definir diccionario de métricas. 2. Priorizar datos de plataforma y tickets antes de añadir SDK. 3. No incluir identificadores personales en informes ordinarios. 4. Revisar sesgos y muestras pequeñas. 5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado. 6. Registrar en el ticket o release record el impacto específico de **Sentimiento** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

140. Reviews

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **reviews** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo. 1. Definir diccionario de métricas. 2. Priorizar datos de plataforma y tickets antes de añadir SDK. 3. No incluir identificadores personales en informes ordinarios. 4. Revisar sesgos y muestras pequeñas. 5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado. 6. Registrar en el ticket o release record el impacto específico de **Reviews** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

141. Wishlists convertidas

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **wishlists convertidas** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual,

QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo. 1. Definir diccionario de métricas. 2. Priorizar datos de plataforma y tickets antes de añadir SDK. 3. No incluir identificadores personales en informes ordinarios. 4. Revisar sesgos y muestras pequeñas. 5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado. 6. Registrar en el ticket o release record el impacto específico de **Wishlists convertidas** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

142. Privacidad de métricas

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **privacidad de métricas** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo. 1. Definir diccionario de métricas. 2. Priorizar datos de plataforma y tickets antes de añadir SDK. 3. No incluir identificadores personales en informes ordinarios. 4. Revisar sesgos y muestras pequeñas. 5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado. 6. Registrar en el ticket o release record el impacto específico de **Privacidad de métricas** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

143. Telemetría opcional

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **telemetría opcional** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe instrumentación. Añadirla implica SDK, flujo de datos, proveedor, consentimiento o base aplicable, seguridad y mantenimiento.

Regla operativa. La telemetría es NOT OPEN hasta completar 26_Privacy_Data_and_Telemetry_Plan.md y una decisión formal.

Procedimiento mínimo. 1. Definir diccionario de métricas. 2. Priorizar datos de plataforma y tickets antes de añadir SDK. 3. No incluir identificadores personales en informes ordinarios. 4. Revisar sesgos y muestras pequeñas. 5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado. 6. Registrar en el ticket o release record el impacto específico de **Telemetría opcional** y su relación con versión, build, save y comunicación.

Evidencia mínima. DPI/data map, contrato, política, opt-out, prueba y aprobación.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

144. No recolectar datos sin política

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **no recolectar datos sin política** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo. 1. Definir diccionario de métricas. 2. Priorizar datos de plataforma y tickets antes de añadir SDK. 3. No incluir identificadores personales en informes ordinarios. 4. Revisar sesgos y muestras pequeñas. 5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado. 6. Registrar en el ticket o release record el impacto específico de **No recolectar datos sin política** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

145. Informes diarios

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **informes diarios** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo. 1. Definir diccionario de métricas. 2. Priorizar datos de plataforma y tickets antes de añadir SDK. 3. No incluir identificadores personales en informes ordinarios. 4. Revisar sesgos y muestras pequeñas. 5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado. 6. Registrar en el ticket o release record el impacto específico de **Informes diarios** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

146. Informes semanales

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **informes semanales** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo. 1. Mantener inventario de dependencias y notices. 2. Probar cambios en rama aislada. 3. Conservar instaladores/manifests y saves de referencia. 4. Definir ventana de mantenimiento y criterios de EOL. 5. Archivar código, builds, documentación y decisiones. 6. Registrar en el ticket o release record el impacto específico de **Informes semanales** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

147. Retrospectiva

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **retrospectiva** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo. 1. Mantener inventario de dependencias y notices. 2. Probar cambios en rama aislada. 3. Conservar instaladores/manifests y saves de referencia. 4. Definir ventana de mantenimiento y criterios de EOL. 5. Archivar código, builds, documentación y decisiones. 6. Registrar en el ticket o release record el impacto específico de **Retrospectiva** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

148. Mantenimiento

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **mantenimiento** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo. 1. Mantener inventario de dependencias y notices. 2. Probar cambios en rama aislada. 3. Conservar instaladores/manifests y saves de referencia. 4. Definir ventana de mantenimiento y criterios de EOL. 5. Archivar código, builds, documentación y decisiones. 6. Registrar en el ticket o release record el impacto específico de **Mantenimiento** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

149. Dependencias

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **dependencias** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo. 1. Mantener inventario de dependencias y notices. 2. Probar cambios en rama aislada. 3. Conservar instaladores/manifests y saves de referencia. 4. Definir ventana de mantenimiento y criterios de EOL. 5. Archivar código, builds, documentación y decisiones. 6. Registrar en el ticket o release record el impacto específico de **Dependencias** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

Los upgrades se realizan en rama aislada, con lockfile, changelog, licencia/NOTICE, build y regresión antes de decidir adopción.

150. Unity y paquetes

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **unity y paquetes** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo. 1. Mantener inventario de dependencias y notices. 2. Probar cambios en rama aislada. 3. Conservar instaladores/manifests y saves de referencia. 4. Definir ventana de mantenimiento y criterios de EOL. 5. Archivar código, builds, documentación y decisiones. 6. Registrar en el ticket o release record el impacto específico de **Unity y paquetes** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

Los upgrades se realizan en rama aislada, con lockfile, changelog, licencia/NOTICE, build y regresión antes de decidir adopción.

151. Vulnerabilidades

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **vulnerabilidades** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo. 1. Mantener inventario de dependencias y notices. 2. Probar cambios en rama aislada. 3. Conservar instaladores/manifests y saves de referencia. 4. Definir ventana de mantenimiento y criterios de EOL. 5. Archivar código, builds, documentación y decisiones. 6. Registrar en el ticket o release record el impacto específico de **Vulnerabilidades** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

Los upgrades se realizan en rama aislada, con lockfile, changelog, licencia/NOTICE, build y regresión antes de decidir adopción.

152. Actualizaciones del motor

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **actualizaciones del motor** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo. 1. Mantener inventario de dependencias y notices. 2. Probar cambios en rama aislada. 3. Conservar instaladores/manifests y saves de referencia. 4. Definir ventana de mantenimiento y criterios de EOL. 5. Archivar código, builds, documentación y decisiones. 6. Registrar en el ticket o release record el impacto específico de **Actualizaciones del motor** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

Los upgrades se realizan en rama aislada, con lockfile, changelog, licencia/NOTICE, build y regresión antes de decidir adopción.

153. Compatibilidad futura

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **compatibilidad futura** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo. 1. Mantener inventario de dependencias y notices. 2. Probar cambios en rama aislada. 3. Conservar instaladores/manifests y saves de referencia. 4. Definir ventana de mantenimiento y criterios de EOL. 5. Archivar código, builds, documentación y decisiones. 6. Registrar en el ticket o release record el impacto específico de **Compatibilidad futura** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

El diagnóstico compara hardware, OS, drivers, resolución, branch y build; se evita atribuir el problema a la plataforma sin reproducción controlada.

154. Pruebas de regresión

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **pruebas de regresión** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo. 1. Mantener inventario de dependencias y notices. 2. Probar cambios en rama aislada. 3. Conservar instaladores/manifests y saves de referencia. 4. Definir ventana de mantenimiento y criterios de EOL. 5. Archivar código, builds, documentación y decisiones. 6. Registrar en el ticket o release record el impacto específico de **Pruebas de regresión** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

155. Deuda técnica

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **deuda técnica** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo. 1. Mantener inventario de dependencias y notices. 2. Probar cambios en rama aislada. 3. Conservar instaladores/manifests y saves de referencia. 4. Definir ventana de mantenimiento y criterios de EOL. 5. Archivar

código, builds, documentación y decisiones. 6. Registrar en el ticket o release record el impacto específico de **Deuda técnica** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

156. End of life

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **end of life** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El fin de vida no es abandono repentino; es una transición gobernada. Para un juego offline, la última build debe seguir siendo utilizable sin servicios innecesarios.

Regla operativa. Definir condiciones, periodo de aviso, soporte de saves, retirada de integraciones y archivo.

Procedimiento mínimo. 1. Mantener inventario de dependencias y notices. 2. Probar cambios en rama aislada. 3. Conservar instaladores/manifests y saves de referencia. 4. Definir ventana de mantenimiento y criterios de EOL. 5. Archivar código, builds, documentación y decisiones. 6. Registrar en el ticket o release record el impacto específico de **End of life** y su relación con versión, build, save y comunicación.

Evidencia mínima. Plan EOL, aviso, build final, saves, notices y archivo.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

157. Fin de soporte

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **fin de soporte** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Se diferencia de deslistado y de fin de venta. Puede cesar soporte activo manteniendo descarga y juego offline.

Regla operativa. No anunciarlo hasta decidir fechas, canales, excepciones de seguridad y destino de datos/tickets.

Procedimiento mínimo. 1. Mantener inventario de dependencias y notices. 2. Probar cambios en rama aislada. 3. Conservar instaladores/manifests y saves de

referencia. 4. Definir ventana de mantenimiento y criterios de EOL. 5. Archivar código, builds, documentación y decisiones. 6. Registrar en el ticket o release record el impacto específico de **Fin de soporte** y su relación con versión, build, save y comunicación.

Evidencia mínima. Aviso multilingüe, calendario, FAQ y archivo.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

158. Aviso a jugadores

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **aviso a jugadores** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo. 1. Mantener inventario de dependencias y notices. 2. Probar cambios en rama aislada. 3. Conservar instaladores/manifests y saves de referencia. 4. Definir ventana de mantenimiento y criterios de EOL. 5. Archivar código, builds, documentación y decisiones. 6. Registrar en el ticket o release record el impacto específico de **Aviso a jugadores** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

159. Conservación de saves

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **conservación de saves** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios

online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo. 1. Mantener inventario de dependencias y notices. 2. Probar cambios en rama aislada. 3. Conservar instaladores/manifests y saves de referencia. 4. Definir ventana de mantenimiento y criterios de EOL. 5. Archivar código, builds, documentación y decisiones. 6. Registrar en el ticket o release record el impacto específico de **Conservación de saves** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

Las pruebas usan fixtures reales anonimizados, primary/backup y estados límite; nunca trabajan sobre la única copia del jugador.

160. Disponibilidad offline

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **disponibilidad offline** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El diseño actual es local/offline y no integra autenticación o servidores. Esa propiedad debe preservarse salvo decisión explícita.

Regla operativa. No introducir dependencia de red para iniciar, cargar, guardar o jugar el contenido base.

Procedimiento mínimo. 1. Mantener inventario de dependencias y notices. 2. Probar cambios en rama aislada. 3. Conservar instaladores/manifests y saves de referencia. 4. Definir ventana de mantenimiento y criterios de EOL. 5. Archivar código, builds, documentación y decisiones. 6. Registrar en el ticket o release record el impacto específico de **Disponibilidad offline** y su relación con versión, build, save y comunicación.

Evidencia mínima. Prueba sin conexión desde instalación limpia y tras reinicio.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

161. Archivado

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **archivado** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo. 1. Mantener inventario de dependencias y notices. 2. Probar cambios en rama aislada. 3. Conservar instaladores/manifests y saves de referencia. 4. Definir ventana de mantenimiento y criterios de EOL. 5. Archivar código, builds, documentación y decisiones. 6. Registrar en el ticket o release record el impacto específico de **Archivado** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

162. Work packages

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **work packages** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo. 1. Mantener inventario de dependencias y notices. 2. Probar cambios en rama aislada. 3. Conservar instaladores/manifests y saves de referencia. 4. Definir ventana de mantenimiento y criterios de EOL. 5. Archivar

código, builds, documentación y decisiones. 6. Registrar en el ticket o release record el impacto específico de **Work packages** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

ID	Paquete	Gate
OPS-WP-01	Canales y soporte público	Pre-Coming Soon
OPS-WP-02	Incident register y plantillas	H6+
OPS-WP-03	Save migration framework	Pre-release
OPS-WP-04	Steam branches y rollback drill	Release candidate
OPS-WP-05	Patch/hotfix pipeline	Release candidate
OPS-WP-06	Community rules/moderation	Coming Soon
OPS-WP-07	Patch notes ES/EN	Pre-release
OPS-WP-08	Crash evidence strategy	Pre-release
OPS-WP-09	Cloud feasibility and conflict tests	Optional feature gate
OPS-WP-10	Metrics/privacy decision	Post-privacy-plan
OPS-WP-11	Maintenance/dependency cadence	Launch
OPS-WP-12	EOL/archive plan	Before end of support

163. Gates

Este capítulo forma parte del bloque **ejecución y gobernanza final** y define cómo se gobierna **gates** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Este plan es preparatorio. Su valor depende de convertir capítulos en work packages, gates, checklists, plantillas y revisiones vivas. Ningún elemento postlanzamiento está operativo por el mero hecho de estar documentado.

Regla operativa. Cada gate necesita owner, evidencia y decisión manual. Las plantillas reducen errores, pero no reemplazan criterio. Los riesgos se revisan tras cada incidente, parche, cambio de Steamworks o modificación sustancial del producto.

Procedimiento mínimo. 1. Asignar work packages y dependencias. 2. Definir gates de Ready/Done. 3. Ensayar checklists en una release interna. 4. Versionar plantillas y glosario. 5. Actualizar historial con motivo, evidencia y rollback. 6. Registrar en el ticket o release record el impacto específico de **Gates** y su relación con versión, build, save y comunicación.

Evidencia mínima. Backlog, matriz de gates, paquete de plantillas y changelog aprobado.

Fuentes Steamworks relacionadas: ST-OPS-015.

164. Checklists

Este capítulo forma parte del bloque **ejecución y gobernanza final** y define cómo se gobierna **checklists** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Este plan es preparatorio. Su valor depende de convertir capítulos en work packages, gates, checklists, plantillas y revisiones vivas. Ningún elemento postlanzamiento está operativo por el mero hecho de estar documentado.

Regla operativa. Cada gate necesita owner, evidencia y decisión manual. Las plantillas reducen errores, pero no reemplazan criterio. Los riesgos se revisan tras cada incidente, parche, cambio de Steamworks o modificación sustancial del producto.

Procedimiento mínimo. 1. Asignar work packages y dependencias. 2. Definir gates de Ready/Done. 3. Ensayar checklists en una release interna. 4. Versionar plantillas y glosario. 5. Actualizar historial con motivo, evidencia y rollback. 6. Registrar en el ticket o release record el impacto específico de **Checklists** y su relación con versión, build, save y comunicación.

Evidencia mínima. Backlog, matriz de gates, paquete de plantillas y changelog aprobado.

Fuentes Steamworks relacionadas: ST-OPS-015.

165. Plantillas

Este capítulo forma parte del bloque **ejecución y gobernanza final** y define cómo se gobierna **plantillas** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Este plan es preparatorio. Su valor depende de convertir capítulos en work packages, gates, checklists, plantillas y revisiones vivas. Ningún

elemento postlanzamiento está operativo por el mero hecho de estar documentado.

Regla operativa. Cada gate necesita owner, evidencia y decisión manual. Las plantillas reducen errores, pero no reemplazan criterio. Los riesgos se revisan tras cada incidente, parche, cambio de Steamworks o modificación sustancial del producto.

Procedimiento mínimo. 1. Asignar work packages y dependencias. 2. Definir gates de Ready/Done. 3. Ensayar checklists en una release interna. 4. Versionar plantillas y glosario. 5. Actualizar historial con motivo, evidencia y rollback. 6. Registrar en el ticket o release record el impacto específico de **Plantillas** y su relación con versión, build, save y comunicación.

Evidencia mínima. Backlog, matriz de gates, paquete de plantillas y changelog aprobado.

Fuentes Steamworks relacionadas: ST-OPS-015.

166. Riesgos

Este capítulo forma parte del bloque **ejecución y gobernanza final** y define cómo se gobierna **riesgos** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Los riesgos principales son save corruption, rollback incompatible, actualización defectuosa, soporte desbordado, moderación reactiva, promesas públicas, Cloud prematuro, privacidad, paquetes y pérdida de trazabilidad.

Regla operativa. Cada riesgo tiene probabilidad, impacto, trigger, owner, mitigación, contingencia y evidencia de cierre.

Procedimiento mínimo. 1. Asignar work packages y dependencias. 2. Definir gates de Ready/Done. 3. Ensayar checklists en una release interna. 4. Versionar plantillas y glosario. 5. Actualizar historial con motivo, evidencia y rollback. 6. Registrar en el ticket o release record el impacto específico de **Riesgos** y su relación con versión, build, save y comunicación.

Evidencia mínima. Registro de riesgos actualizado después de cada release o incidente.

Fuentes Steamworks relacionadas: ST-OPS-015.

167. Glosario

Este capítulo forma parte del bloque **ejecución y gobernanza final** y define cómo se gobierna **glosario** después de una publicación. La regla se aplica tanto

a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Este plan es preparatorio. Su valor depende de convertir capítulos en work packages, gates, checklists, plantillas y revisiones vivas. Ningún elemento postlanzamiento está operativo por el mero hecho de estar documentado.

Regla operativa. Cada gate necesita owner, evidencia y decisión manual. Las plantillas reducen errores, pero no reemplazan criterio. Los riesgos se revisan tras cada incidente, parche, cambio de Steamworks o modificación sustancial del producto.

Procedimiento mínimo. 1. Asignar work packages y dependencias. 2. Definir gates de Ready/Done. 3. Ensayar checklists en una release interna. 4. Versionar plantillas y glosario. 5. Actualizar historial con motivo, evidencia y rollback. 6. Registrar en el ticket o release record el impacto específico de **Glosario** y su relación con versión, build, save y comunicación.

Evidencia mínima. Backlog, matriz de gates, paquete de plantillas y changelog aprobado.

Fuentes Steamworks relacionadas: ST-OPS-015.

168. Historial de cambios

Este capítulo forma parte del bloque **ejecución y gobernanza final** y define cómo se gobierna **historial de cambios** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Este plan es preparatorio. Su valor depende de convertir capítulos en work packages, gates, checklists, plantillas y revisiones vivas. Ningún elemento postlanzamiento está operativo por el mero hecho de estar documentado.

Regla operativa. Cada gate necesita owner, evidencia y decisión manual. Las plantillas reducen errores, pero no reemplazan criterio. Los riesgos se revisan tras cada incidente, parche, cambio de Steamworks o modificación sustancial del producto.

Procedimiento mínimo. 1. Asignar work packages y dependencias. 2. Definir gates de Ready/Done. 3. Ensayar checklists en una release interna. 4. Versionar plantillas y glosario. 5. Actualizar historial con motivo, evidencia y rollback. 6. Registrar en el ticket o release record el impacto específico de **Historial de cambios** y su relación con versión, build, save y comunicación.

Evidencia mínima. Backlog, matriz de gates, paquete de plantillas y changelog aprobado.

Fuentes Steamworks relacionadas: ST-OPS-015.

Anexo A. Fuentes oficiales de Steamworks verificadas

Las siguientes páginas se verificaron el **1 de julio de 2026**. Se conservan como fuentes operativas, pero deben revalidarse en Steamworks antes de activar una feature o cambiar un proceso.

ID	Fuente	URL	Uso
ST-OPS-001	Steam Community	https://partner.steamgames.com/doc/features/community	Humanesitability, developer identity, forums and moderation responsibilities.
ST-OPS-002	Community Moderation	https://partner.steamgames.com/doc/marketing/community_moderation	Flagging, off/online responses, moderator practice and escalation to Steamworks Support.
ST-OPS-003	User Reviews	https://partner.steamgames.com/doc/store/reviews	Reviews rules, developer responses, review-bomb escalation and feedback interpretation.
ST-OPS-004	Updating Game Build	https://partner.steamgames.com/doc/sdk/updating	Uploads, create a test branch before moving it to default.
ST-OPS-005	Branches (Betas)	https://partner.steamgames.com/doc/store/application/branches	Branches, client opt-in, default branch behavior and test workflow.
ST-OPS-006	Builds and Manifests	https://partner.steamgames.com/doc/store/application/builds	Builds, depot, manifest model, default authorization and beta branches.

ID	Fuente	URL	Uso
ST-OPS-007	Uploading to Steam	https://partner.steamgames.com/doc/sdk/uploading	Steamworks ContentBuilder, builds, depots and rollback to previous uploads.
ST-OPS-008	Steam Cloud	https://partner.steamgames.com/doc/features/cloud	AgarCloud/ABI, per-user paths, quotas, developer-only testing, dynamic sync and cloud logs.
ST-OPS-009	Small Update / Patch Notes	https://partner.steamgames.com/doc/marketing/event_tools/type_	AgarStore/ABI, type and communication expectations for every pushed build.
ST-OPS-010	Events and Announcements	https://partner.steamgames.com/doc/marketing/event_tools	Library/store/Account/marketing, communication for patches and updates.
ST-OPS-011	Updating Your Game — Best Practices	https://partner.steamgames.com/doc/store/updates	Games, announcements and major-update visibility guidance.
ST-OPS-012	Steam Error Reporting	https://partner.steamgames.com/doc/features/error_reporting	Legacy crash reporting is nearing end-of-life and limited to Windows 32-bit.
ST-OPS-013	Developer Refund Reporting	https://partner.steamgames.com/doc/finance/refunds	Refunds, voluntary and player-provided reasons in Steam financial reports.
ST-OPS-014	Managing Steamworks Users	https://partner.steamgames.com/doc/gettingstarted/managing_user	Users, groups, permissions and moderator access.

ID	Fuente	URL	Uso
ST-OPS-015	Contacting Steamworks Support	https://partner.steamgames.com/doc/help/contact	Official support or escalation route for partner issues.

Anexo B. Evidencia de implementación observada

Evidencia	Estado
Producto/versión	Cartridge & Cloud / 0.0.21
Empresa configurada	VRM Games
Resolución configurada	1024×768, resizable 0
Guardado integrado	JSON, primary + backup + temp + recovery, generación y validación
Schema	IntegratedGameStateSnapshot.CurrentSchemaVersion = 2
Steamworks	no encontrado en código/paquetes
Steam Cloud	no encontrado
Telemetría/analytics	no encontrado
Crash SDK externo	no encontrado
HTTP/network client de producto	no encontrado
Paquetes directos	41

Escenas de build observadas

Enabled	Escena	GUID
sí	Assets/_Project/Scenes/BuildStateSnapshot	8a5408551a6e091b9468e2109ccc7cbfc0
sí	Assets/_Project/Scenes/BuildStateSnapshot	0a1786c6952a55d418cf16503030c766
sí	Assets/_Project/Scenes/BuildStateSnapshot	55a7ce906944f74bad3a0fb7d62f8e5
sí	Assets/_Project/Scenes/BuildStateSnapshot	0e212ad62a13c614b91e32722c9452be

Archivos técnicos hashados

Archivo	Bytes	SHA-256
Assets/_Project/Scripts/Infrastructure/Persistence/IntegratedStateSnapshot	3428	8a5408551a6e091b9468e2109ccc7cbfc0
Assets/_Project/Scripts/Infrastructure/Persistence/IntegratedStateSnapshot	1536	0a1786c6952a55d418cf16503030c766
Assets/_Project/Scripts/Infrastructure/UI/BuildStateSnapshot	2065	55a7ce906944f74bad3a0fb7d62f8e5
Assets/_Project/Scripts/Application/Persistence/IntegratedStateSnapshot	3093	0e212ad62a13c614b91e32722c9452be

Archivo	Bytes	SHA-256
Assets/_Project/Scripts/Domain/Persistence/00_Integration/02_Build/03_Data/04_Scene/05_0104430e830b9ed2c69a	20611	9fa497022b2f54ac5370f5fa90bf1
Assets/_Project/Scripts/Infrastructure/GameSession/04_Build/05_Game/06_Pipeline/07_05307bbd8143c00	16882	4a41b006879f70e7241815f095691b
Assets/_Project/Settings/BuildProfiles/Windows/06_Editor/03_038944a84c37fc636fa908e9d8fbc44d	946	6a9e58288030c38944a84c37fc636fa908e9d8fbc44d
Assets/_Project/Scenes/StoreInitial.unity	90851	ca9689130ae8ad29a04af7145864451213a81d60c249
Assets/_Project/Scenes/Store.unity	88611	f8ceebcb55d1a97ca0a1205363d31f8641107e52968b
Assets/_Project/Scenes/TestLab.unity	29023	b390fbb375d5f9fb97563afb01fc3e6c0e612b446341

Anexo C. Inventario de documentos consolidados

Documento	Bytes	SHA-256
00_Enfoque_y_Alcance.md	127401	63dd6f0f1b9c2c80be2aec55718d18d9d031ce4acb14
01_Game_Design_Document.md	132822	a9cd9e3203abc5269c25c59dc7f626b0771df4ec2d65
02_Vertical_Slice_Specification.md	64531	75893f130116e88a08b8da5590dd81876a234581081e
02_Vertical_Slice_Specification.preview.md	64531	75893f130116e88a08b8da5590dd81876a234581081e
03_Technical_Design_Document.draft.md	101994	66264703c5ff4959d03093690e9845a98ec0e5025e68
03_Technical_Design_Document.md	101994	66264703c5ff4959d03093690e9845a98ec0e5025e68
04_Modelo_de_Datos.draft.md	102948	d851a72b55742c2c704cc7c002929bc5894e7142511c
04_Modelo_de_Datos.md	102948	d851a72b55742c2c704cc7c002929bc5894e7142511c
05_UX_Flow.md	56805	e08da5ff67fb073a8b950babe325ae58ddb2e121513d
06_Production_Roadmap_y_Sprint_Plan.md	61564	d00b156c0ad1c66069278119c55f76c738a577a3e283
07_QA_Testing_Plan.md	79435	bc9c05a3737e345546ef16e5390e034295ef35ad66c3
08_QA_Testing_Matrix.xlsx	185032	233e57f01d2468fe13c136c8fcd3ee75b39bd3de6530
09_CSharp_Coding_Standards.md	95321	3b734d6cd49f31c09c10bb4941625e143f6c634e3fb5
10_Unity_Project_Setup_Guide.md	78408	31430bb0544edd9577323aa0009d6ca3df8b9fbcf6c8
11_Build_y_Versioning_Guide.md	85324	38a16a0f97505c0b405509dd3c7e3d1b50e77b89fbe3
12_Excel_Maestro_de_Produccion.xlsx	158100	405376cb64f49b34f1f842f3840c12654e9f08e2ec45
13_Trazabilidad_y_Control_de_Cambios.xlsx	189424	f674e21d1a3a1f0970a3d339f26f2a2b51a19b5c257c
14_Project_Binder_Indice_Maestro.md	92828	0c24dcd47e0d44793e75e764e3fc2c146e5b1fea04d8
15_Guia_Maestra.md	119920	879bc1925cccf77e10df6c104302f3afc72f71e8c58d
16_Auditoria_Global_de_Coherencia.md	87439	4a62376b8aee623be40957ae0511d2dcab2919947f3a
17_Art_Bible.md	133913	095c6cc42223a972c7faa68435595fc7c22f839fabc6
18_Audio_Bible.md	144525	c2eabe93a7c26f70e1d4d77fb0ddcaa97e2cdd1e7024
19_UI_Style_Guide.md	149800	b34ca4db1eba7a536466b6a9a105d8d92396ade2108a
20_Economy_and_Balance_Specification.md	154547	908c70772ed35eecd05c3036c6901523a4caf2c3e70
21_Initial_Content_Catalog.xlsx	174383	9047299f60cfbd368fc4a2c8bee8d0ff4132dc564236
22_Localization_Plan.md	157323	184a53b2813bc1327a6913e6e88ea4c81142da1ee4dd
23_Legal_Credits_and_Licenses_Register.xlsx	175135	f543b3b5de8900f405910fac5d72697045f8c4969d98
24_Steam_Publishing_Plan.md	171037	f087e24d3a18f8c4a6651728d5f70d1df804042587e2

Anexo D. Inventario histórico y operativo preservado

Se inventarían **241** fuentes relacionadas con release, QA, builds, soporte, incidentes, persistencia, Steam y gobernanza. No se omiten las versiones anteriores: sus hashes permiten distinguir reediciones idénticas y cambios reales.

[illegible]

[illegible]

Baseline/área	Tipo	Fuente	Bytes	SHA-256
0.5	MD	Documentation/00_Official	4120	Base4172846408569335055a2e76ae56d10467ad1762d
0.5	MD	Documentation/00_Official	2100	Base4Bff78b69a569a0e80f87834a828590d642ad1d06d
0.5	MD	Documentation/00_Official	189	Base011a3e1b0286f99758449c85b4a0c34ea607a6f2ba
0.5	MD	Documentation/00_Official	319	Basee557e3004a8f99572a6bfcd4e6240b23860ba4f6ba
0.5	MD	Documentation/00_Official	365	Base0623a03b2f99758a95ee06babe45f0c6a0f6f6
0.5	MD	Documentation/00_Official	285	Base19ab67f2669e43e44fcd5d6ad481f0d163adbf0
0.5	MD	Documentation/00_Official	2102	Base4966404092e989898c5bd87483a0b10a6a6f4d
0.5	MD	Documentation/00_Official	414	Base2067e160a509a0e94acc6996b37d0ca234c4ada
0.5	MD	Documentation/00_Official	281	Base4721e3407b539f75b095f6e38a44d00a6c679ee1d
0.5	MD	Documentation/00_Official	27	Base0c25400f3e145844f5b235a5f02b346b02f564ad
0.6	PDF	Documentation/00_Official	375	Base144b04583d1601b0924288c0a9b2f55e87a9b771c
0.6	PDF	Documentation/00_Official	1315	Base47002060df83e5e854263a4a6028240ed1451
0.6	PDF	Documentation/00_Official	385	Base267e20d1a0011B044208340a10279566f81d9991
0.6	PDF	Documentation/00_Official	370	Base7583872f65f8e217e00c102b557c7ac5b9f2eab0
0.6	PDF	Documentation/00_Official	380	Base296fb6062600b5c440a1d4f5c98b7c7d0e1404
0.6	PDF	Documentation/00_Official	4159	Base471b69c540b322B55ff767b5c6ef1cb3b0e0f4ac
0.6	PDF	Documentation/00_Official	371	Basee1ba0f6a9e45473d252102baba7f54a7761134e
0.6	XLSX	Documentation/00_Official	81	Base6ba6eb82726249342H7d1d17140661a0b0d7709e
0.6	PDF	Documentation/00_Official	3829	Base4f45d4a919545374e6b0d2a44e0f07c6e65b39d
0.6	PDF	Documentation/00_Official	2875	Basee1e92636089442505e5a6292b440b6c2a7f981
0.6	PDF	Documentation/00_Official	2878	Base0a7ee40cfb6039f46a303f93f8e8e65c073e5d
0.6	PDF	Documentation/00_Official	2873	Base211ea9a0e670310825f734a2525b60a8e070a67e1
0.6	MD	Documentation/00_Official	14	Base7b297080a360e00e4d290a63d1f0e3a0bf8382
0.6	MD	Documentation/00_Official	89	Base010a50b839062087a3630b810c065431d75d11af
0.6	MD	Documentation/00_Official	78	Base9f0a82104603238f0b8700e0223830006e24b8a
0.6	MD	Documentation/00_Official	4	Basef67md95b0c62bd1ef61100f0e538e0a2992650r
0.6	MD	Documentation/00_Official	6	Base0c336c80f652694e0d4b028a0c0f4e0e80b839a
0.6	MD	Documentation/00_Official	5	Base011a3e1b0286f99758a95ee06babe45f0c6a0f6f6
0.6	MD	Documentation/00_Official	7	Base4b0529a0e64f67e2a6a700f0e01b02f0f0f0f0f0
0.6	MD	Documentation/00_Official	32	Base4016e6e316800500f5e7d1p0f0e160e0a0b0f34r
0.6	CSV	Documentation/00_Official	16	Base263a650ade016a345e8b120e99c294aee975135r
0.6	MD	Documentation/00_Official	23	Basefb2nd4557e306d07654970d45f455e4e0a0303r
0.6	MD	Documentation/00_Official	216	Base0c00f62e233e9f180d2be034071d2069f6e2e0a61d
0.6	MD	Documentation/00_Official	256	Base0f1ba9380260c99833003b20f0a0b79002842d61d3
0.6	MD	Documentation/00_Official	40	Base3B077705840e03e29cd04a0f00f0464a6011d
0.6	MD	Documentation/00_Official	15	Base10fe20a16080a31050f677400839e50f62a161d0
0.6	MD	Documentation/00_Official	89	BaseB063e08a67935087f5dce4d078b94a008620bd
0.6	MD	Documentation/00_Official	316	Base4092a1b096009734eb05a05a380600346e00e10
0.6	MD	Documentation/00_Official	319	Base471b69c540b322B55ff767b5c6ef1cb3b0e0f4ac
0.6	MD	Documentation/00_Official	263	Base7559a770a56a973f0883d8a0ab5770883f6ae094B
0.6	MD	Documentation/00_Official	86	Base471b69c540b322B55ff767b5c6ef1cb3b0e0f4ac
0.6	MD	Documentation/00_Official	126	Base4b0529a0e64f67e2a6a700f0e01b02f0f0f0f0
0.6	MD	Documentation/00_Official	30	Base1f0f0e01b02f0f0f0f0f0f0f0f0f0f0f0f0f0f0
0.6	MD	Documentation/00_Official	2	Base7b297080a360e00e4d290a63d1f0e3a0bf8382

Baseline/área	Tipo	Fuente	Bytes	SHA-256
working/current records	PNG	Documentation/01_Working_Documents/29jun2026_21_37_54.png	107793	49b742396a4dff3b401ee6a8d7100
working/current records	FILE	Documentation/01_Working_Documents/29jun2026_21_37_54.png	31043298	1164aaf13468996c1924276e1
working/current records	PNG	Documentation/01_Working_Documents/29jun2026_21_38_14(10).png	176773	49175f3a6940b64fb5317038a5dff40463
working/current records	PNG	Documentation/01_Working_Documents/29jun2026_21_38_14(7).png	179316	6150823861c5067943073370e18a64be62
working/current records	PNG	Documentation/01_Working_Documents/29jun2026_21_38_14(8).png	199642	79ef6c5861c5067943073370e18a64be62
working/current records	PNG	Documentation/01_Working_Documents/29jun2026_21_38_14(9).png	175614	1ef9403841c6c164d77a35a1c36cd1904bf1702
working/current records	PNG	Documentation/01_Working_Documents/29jun2026_21_38_14(1).png	199353	410d9f5861c5067943073370e18a64be62
working/current records	PNG	Documentation/01_Working_Documents/29jun2026_21_38_14(2).png	178699	6407c48d119514b64a5343d710b224c0a8b9603
working/current records	PNG	Documentation/01_Working_Documents/29jun2026_21_38_14(3).png	195490	27639f5594b9b24ffc20403c0e45a68b9013

Baseline/área	Tipo	Fuente	Bytes	SHA-256
working/current records	PNG	Documentation/01_Working_Documents/29jun2026_21_38_14(4).png	1039409	7e4e54826fcd3b7628cf7e704c271a91e687c4021
working/current records	PNG	Documentation/01_Working_Documents/29jun2026_21_38_14(5).png	1001478	481e403262dc1869d99133f24d96b24ec5243
working/current records	PNG	Documentation/01_Working_Documents/29jun2026_21_38_14(6).png	1774018	5689423ff619fb91a5230d0c37698c04ef433
working/current records	PNG	Documentation/01_Working_Documents/29jun2026_21_38_50(1).png	1728315	fe43943d1144c4161aee038d417914c1f76f604
working/current records	PNG	Documentation/01_Working_Documents/29jun2026_21_38_50(2).png	1185378	51bca43f10d16b564793260c61c9b77fa3034
working/current records	PNG	Documentation/01_Working_Documents/29jun2026_21_38_50(3).png	1151401	0931913470d562609622789ab9955d19df7004
working/current records	PNG	Documentation/01_Working_Documents/29jun2026_21_38_50(4).png	1121051	546539311baf0a40f033b4921bd4349a7e674
working/current records	PNG	Documentation/01_Working_Documents/29jun2026_21_38_50(5).png	1154800	1e02633ff9c07b6568b43b6fca9957a11044

Baseline/área	Tipo	Fuente	Bytes	SHA-256
working/current records	PNG	Documentation/01_Working_Documentos/Spine_29_2026, 21_38_50 (6).png	11775107	660416932df54961461562dfc6ff99d03db7854
working/current records	PNG	Documentation/01_Working_Documentos/Spine_29_2026, 21_38_50 (7).png	1184105	1b5495218d6164ac573d7c8bf4db840e964
working/current records	PNG	Documentation/01_Working_Documentos/Spine_29_2026, 21_38_50 (8).png	1002007	3611d8736c690d4675bc08f7c6469e29acc5b04
working/current records	PNG	Documentation/01_Working_Documentos/Spine_29_2026, 21_39_17 (1).png	2230307	28b831e2bf6165ab627d470247e794ac9608
working/current records	PNG	Documentation/01_Working_Documentos/Spine_29_2026, 21_39_17 (2).png	2385231	79e0c5381a812667a63b5f38f0c4ba1ba1b58
working/current records	PNG	Documentation/01_Working_Documentos/Spine_29_2026, 21_39_17 (3).png	2105489	79e0c5381a812667a63b5f38f0c4ba1ba1b58
working/current records	PNG	Documentation/01_Working_Documentos/Spine_29_2026, 21_39_17 (4).png	2065908	1e74b89d1051c0632fa233256049158404679
working/current records	PNG	Documentation/01_Working_Documentos/Spine_29_2026, 21_39_17 (5).png	2108708	626585944b10561a2c3da08297e284dd00f5

Baseline/área	Tipo	Fuente	Bytes	SHA-256
working/current records	PNG	Documentation/01_Working/Current/2026jun21_39_17(6).png	22382016	7e34968719258b8840e723f3529fb4feb605
working/current records	PNG	Documentation/01_Working/Current/2026jun21_39_17(7).png	2038776	c9b8794414a1f61750c3d162b309a6204d298
working/current records	PNG	Documentation/01_Working/Current/2026jun21_39_17(8).png	1062108	16d263a7a016ba78cb53b610b84ab44286805
working/current records	MD	Documentation/10_Development/Records/635b1f238564e9d1h7900b3f155ba2h	1181	2b2d0b635b1f238564e9d1h7900b3f155ba2h
working/current records	MD	Documentation/10_Development/Records/48666443b1bf973b1f361e28452d3ff3010	1168	48666443b1bf973b1f361e28452d3ff3010
working/current records	MD	Documentation/10_Development/Records/5a4e0a192f1e385594582a1d0ee415a170e2a	1155	5a4e0a192f1e385594582a1d0ee415a170e2a
working/current records	MD	Documentation/10_Development/Records/383833b1112a1834518e934c5ee3f1ce23	1065	383833b1112a1834518e934c5ee3f1ce23
working/current records	MD	Documentation/10_Development/Records/4f8e4fda5029fcd60e1970116543f82a	1379	4f8e4fda5029fcd60e1970116543f82a
working/current records	MD	Documentation/10_Development/Records/3e3feda89b37fa856130c1e9255240b18235	1169	3e3feda89b37fa856130c1e9255240b18235
working/current records	MD	Documentation/10_Development/Records/4048b0518e41b0d526f4be10a15ae1f107547e41	2595	4048b0518e41b0d526f4be10a15ae1f107547e41
working/current records	MD	Documentation/10_Development/Records/184f8153441e6052438632676ad5f7a1dd449a	998	184f8153441e6052438632676ad5f7a1dd449a
working/current records	MD	Documentation/10_Development/Records/70ad9d5436b32071269f45d235e7f7e6889b95f	1887	70ad9d5436b32071269f45d235e7f7e6889b95f
working/current records	MD	Documentation/10_Development/Records/1d302783b1bf1c56155c029f460d133731	1629	1d302783b1bf1c56155c029f460d133731
working/current records	MD	Documentation/10_Development/Records/893e627a703f11f882f4db42be29dedu	1398	893e627a703f11f882f4db42be29dedu
working/current records	MD	Documentation/10_Development/Records/3a7538c0b1f4624d613e90cfe51716970a	1648	3a7538c0b1f4624d613e90cfe51716970a
working/current records	MD	Documentation/10_Development/Records/36c0a933c11f62369f283d7e4ed0a07ae	2578	36c0a933c11f62369f283d7e4ed0a07ae
working/current records	MD	Documentation/10_Development/Records/5610c0549e71897049050044b13c03d73737	1186	5610c0549e71897049050044b13c03d73737

Baseline/área	Tipo	Fuente	Bytes	SHA-256
working/current	MD	Documentation/10_Development	1164t	R666215f3b7c1ef7a3030584798c1fdcf729d
records				
working/current	MD	Documentation/10_Development	2209t	B0a6615f380351f0461f33a61c37c1a0857ba
records				
working/current	MD	Documentation/10_Development	2587t	R05c051f38f194404436f34934e0e0a00d
records				
working/current	MD	Documentation/10_Development	1685t	R057814f389918d10960522a7a1446f1f33456
records				
working/current	MD	Documentation/10_Development	1347t	R370019f22261f0905448f36f4d111332f13a
records				
working/current	MD	Documentation/10_Development	2108t	R026519f391183305738f35a024a4c1481f7a
records				
working/current	MD	Documentation/10_Development	2607t	R2e521f3169ef0630af38f39010b0e4924bf
records				
working/current	MD	Documentation/10_Development	1055t	R05e71f38c68c5260f7000283184153f3381b
records				
working/current	MD	Documentation/10_Development	1587t	R2e001f3851461f57f50e9f943ada1f62eb0024
records				
working/current	MD	Documentation/10_Development	3041t	R55fed0386404f02f749f36f7f20d7f41ee27a
records				
working/current	MD	Documentation/10_Development	2007t	R7cd148f382d118a98f11836761435e2baf0d
records				
working/current	MD	Documentation/10_Development	1107t	R2cd6718f39150ef053042e8b42a692837b7b
records				
working/current	MD	Documentation/10_Development	3351t	R551713f232b04f027f638371e47e42e6f41ba
records				
working/current	MD	Documentation/10_Development	2182t	R05696f39956c5307f66f5740b172ef10b1dc
records				
working/current	MD	Documentation/10_Development	2157t	R4f027f38ab1402030a0a95a9069572738a24
records				
working/current	MD	Documentation/10_Development	1297t	R52c345f37607a308f6548520f2e45824a16c
records				
working/current	MD	Documentation/10_Development	1888t	R44b68f3839176108f578f382b4571ca9701ba
records				
working/current	MD	Documentation/10_Development	1581t	R75e35f383811938f64c359adb39f652dec03
records				
working/current	MD	Documentation/10_Development	1071t	R2f955f3811919308f1f5389c56711d417b5
records				
working/current	MD	Documentation/10_Development	1091t	R0c4816f39c14c00f749f331362424089f0b6
records				
working/current	MD	Documentation/10_Development	708t	R00715a3f284fe079f6475b6a6b1a1df63856a
records				
working/current	MD	Documentation/10_Development	101t	R3cd501f3911815499f6b4d42ca2d239a8f1a83
records				

Baseline/área	Tipo	Fuente	Bytes	SHA-256
working/current	MD	Documentation/10_Development	781t	R476834/39fe196239/73b483347899b7a5fe21ba
records				
working/current	MD	Documentation/10_Development	781t	R43c8db/54a2196409/31b139ab3f0b1f6e48d1
records				
working/current	MD	Documentation/10_Development	661t	Re2a208/39b3696019/608/8290f1f0e13a71102
records				
working/current	MD	Documentation/10_Development	781t	Re3a683/39a345654/1e078e2d4b1e17/610eak
records				
working/current	MD	Documentation/10_Development	181t	Re7898t/39fe196239/73b483347899b7a5fe21ba
records				
working/current	MD	Documentation/10_Development	661t	Re35583/39b3696019/608/8290f1f0e13a71102
records				
working/current	MD	Documentation/10_Development	581t	Re4a917/39a345654/1e078e2d4b1e17/610eak
records				
working/current	MD	Documentation/10_Development	541t	Re45f1b/39a345654/1e078e2d4b1e17/610eak
records				
working/current	MD	Documentation/10_Development	581t	Re4a917/39a345654/1e078e2d4b1e17/610eak
records				
working/current	MD	Documentation/10_Development	181t	Re3a683/39a345654/1e078e2d4b1e17/610eak
records				
working/current	MD	Documentation/10_Development	661t	Re35583/39b3696019/608/8290f1f0e13a71102
records				
working/current	MD	Documentation/10_Development	181t	Re3a683/39a345654/1e078e2d4b1e17/610eak
records				
working/current	MD	Documentation/10_Development	151t	Re99346a/39a005447f/932/431921575e0e8bfe0a
records				
working/current	MD	Documentation/10_Development	181t	Re0156f/39a005447f/932/431921575e0e8bfe0a
records				
working/current	MD	Documentation/10_Development	361t	Re83e99d/39a005447f/932/431921575e0e8bfe0a
records				
working/current	MD	Documentation/10_Development	661t	Re416f8a/39a005447f/932/431921575e0e8bfe0a
records				
working/current	MD	Documentation/10_Development	181t	Re35583/39b3696019/608/8290f1f0e13a71102
records				
working/current	MD	Documentation/10_Development	181t	Re3a683/39a345654/1e078e2d4b1e17/610eak
records				
working/current	MD	Documentation/10_Development	621t	Re45a86/39a345654/1e078e2d4b1e17/610eak
records				
working/current	MD	Documentation/10_Development	181t	Re91a83/39a005447f/932/431921575e0e8bfe0a
records				
working/current	MD	Documentation/10_Development	661t	Re7fd34e/39a005447f/932/431921575e0e8bfe0a
records				
working/current	MD	Documentation/10_Development	181t	Re87282a/39a005447f/932/431921575e0e8bfe0a
records				

Baseline/área	Tipo	Fuente	Bytes	SHA-256
working/current	MD	Documentation/10_Development	160t	Recc539a/33691inf_4b98f230d2613a0e3/479fa80
records				
working/current	MD	Documentation/10_Development	108t	Recc539a/33691inf_4b98f230d2613a0e3/479fa80
records				
working/current	MD	Documentation/10_Development	50t	Recc539a/33691inf_4b98f230d2613a0e3/479fa80
records				
working/current	MD	Documentation/10_Development	187t	Recc539a/33691inf_4b98f230d2613a0e3/479fa80
records				
working/current	MD	Documentation/10_Development	76t	Recc539a/33691inf_4b98f230d2613a0e3/479fa80
records				
working/current	MD	Documentation/10_Development	10t	Recc539a/33691inf_4b98f230d2613a0e3/479fa80
records				
working/current	MD	Documentation/10_Development	10t	Recc539a/33691inf_4b98f230d2613a0e3/479fa80
records				
working/current	MD	Documentation/10_Development	20t	Recc539a/33691inf_4b98f230d2613a0e3/479fa80
records				
working/current	MD	Documentation/10_Development	127t	Recc539a/33691inf_4b98f230d2613a0e3/479fa80
records				
working/current	MD	Documentation/10_Development	74t	Recc539a/33691inf_4b98f230d2613a0e3/479fa80
records				
working/current	MD	Documentation/10_Development	25t	Recc539a/33691inf_4b98f230d2613a0e3/479fa80
records				
working/current	MD	Documentation/10_Development	278t	Recc539a/33691inf_4b98f230d2613a0e3/479fa80
records				
working/current	MD	Documentation/10_Development	64t	Recc539a/33691inf_4b98f230d2613a0e3/479fa80
records				
working/current	MD	Documentation/10_Development	24t	Recc539a/33691inf_4b98f230d2613a0e3/479fa80
records				
working/current	MD	Documentation/10_Development	282t	Recc539a/33691inf_4b98f230d2613a0e3/479fa80
records				
working/current	MD	Documentation/10_Development	25t	Recc539a/33691inf_4b98f230d2613a0e3/479fa80
records				
working/current	MD	Documentation/10_Development	258t	Recc539a/33691inf_4b98f230d2613a0e3/479fa80
records				
working/current	MD	Documentation/10_Development	228t	Recc539a/33691inf_4b98f230d2613a0e3/479fa80
records				
working/current	MD	Documentation/10_Development	228t	Recc539a/33691inf_4b98f230d2613a0e3/479fa80
records				
working/current	MD	Documentation/10_Development	277t	Recc539a/33691inf_4b98f230d2613a0e3/479fa80
records				
working/current	MD	Documentation/10_Development	84t	Recc539a/33691inf_4b98f230d2613a0e3/479fa80
records				
working/current	MD	Documentation/10_Development	10t	Recc539a/33691inf_4b98f230d2613a0e3/479fa80
records				

Baseline/área	Tipo	Fuente	Bytes	SHA-256
working/current	MD	Documentation/10_Development	874t	4b5190d1f36111180699f86c1b05fed94a889
records				
working/current	MD	Documentation/10_Development	569t	64944b609960252163368ab545050989a4a
records				
working/current	MD	Documentation/10_Development	1180t	16d2ca429871e0f1e09391b094f5169729a
records				
working/current	MD	Documentation/10_Development	148t	8e441b467eb14af602f74397b2a8f31f63e59
records				
working/current	MD	Documentation/10_Development	886t	f335438f3141f45a162904582664e84860f1d
records				
working/current	MD	Documentation/10_Development	646t	8e85fd57f39c1e905c69a2eb245a5016caf8
records				
working/current	CSV	Documentation/10_Development	6002t	9d16467636c8b071675fa4bf3568590380a
records				
working/current	CSV	Documentation/10_Development	1685t	1d55da79ed40b11667067aa05eb3569ad515f
records				

Anexo E. Matriz de preparación postlanzamiento

Dominio	Ready	Done	Estado actual
Soporte	canales, owner, privacidad, plantillas	simulacro completo y capacidad real	NOT READY
Incidentes	severidades, register, evidence	S0/S1 drill ejecutado	PARTIAL
Hotfix	branch, versioning, tests	hotfix interno publicado y rollback probado	NOT READY
Save migration	schemas y fixtures	migración desde todas las versiones	NOT READY
Steam branches	AppID, depots, permisos	qa/rc/previous_stable verificadas	NOT OPEN
Comunidad	Hub, reglas, moderadores	prueba de publicación/moderación	NOT OPEN
Métricas	diccionario y privacidad	informes reproducibles sin datos excesivos	NOT OPEN
Mantenimiento	dependencias y cadence	primer ciclo de patch cerrado	PLANNED

Dominio	Ready	Done	Estado actual
EOL	criterios y archivo	aviso y build final preservados	FUTURE